Simple Dictionary

Full line-by-line code walkthrough

Phone edition • Source snapshot: 28 October 2026

How to use this document

Each source line has its own label, code and explanation. This edition includes imports, blank lines and closing brackets too.

Code is above its explanation rather than beside it. Leading indentation is removed and long code is wrapped to fit a phone. A wrapped code block is still one original source line.

Line numbers refer to the HTML walkthrough snapshot. Later source edits can change them. The existing API key is hidden in the guide only; app source is unchanged.

Start with BeginnerGuide.pdf for the overall search, favorite and preference flows. Use this document when you want to understand a particular file or line.

The file list below and PDF bookmarks help navigation. Some platform/Room/browser setup goes beyond the modules' short examples. Generated files, binary assets and Xcode project internals are outside the handwritten-source scope.

This volume: 58 files; 2237 annotated source lines.

Files in reading order

1. shared/src/commonMain/kotlin/com/example/dictionary/domain/Models.kt
2. shared/src/commonMain/kotlin/com/example/dictionary/presentation/UiState.kt
3. shared/src/commonMain/kotlin/com/example/dictionary/domain/DictionaryRepository.kt
4. shared/src/commonMain/kotlin/com/example/dictionary/domain/SearchWordUseCase.kt
5. shared/src/commonMain/kotlin/com/example/dictionary/presentation/DictionaryViewModel.kt
6. shared/src/commonMain/kotlin/com/example/dictionary/data/DictionaryRepository.kt
7. shared/src/commonMain/kotlin/com/example/dictionary/App.kt
8. shared/src/commonMain/kotlin/com/example/dictionary/ui/InputScreen.kt

9. shared/src/commonMain/kotlin/com/example/dictionary/ui/ResultScreen.kt
10. shared/src/commonMain/kotlin/com/example/dictionary/ui/HistoryScreen.kt
11. shared/src/commonMain/kotlin/com/example/dictionary/domain/HistoryUseCase.kt
12. shared/src/commonMain/kotlin/com/example/dictionary/domain/PreferencesRepository.kt
13. shared/src/commonMain/kotlin/com/example/dictionary/domain/SlangPreferenceUseCase.kt
14. shared/src/commonMain/kotlin/com/example/dictionary/data/local/HistoryEntity.kt
15. shared/src/commonMain/kotlin/com/example/dictionary/data/local/HistoryDao.kt
16. shared/src/commonMain/kotlin/com/example/dictionary/data/local/DictionaryDatabase.kt
17. shared/src/commonMain/kotlin/com/example/dictionary/data/DictionaryPreferences.kt
18. shared/src/commonMain/kotlin/com/example/dictionary/data/AppStorage.kt
19. shared/src/commonMain/kotlin/com/example/dictionary/data/HttpClientFactory.kt
20. androidApp/build.gradle.kts
21. androidApp/src/main/AndroidManifest.xml
22. androidApp/src/main/kotlin/com/example/dictionary/MainActivity.kt
23. androidApp/src/main/res/values/strings.xml
24. browserStorage/build.gradle.kts
25. browserStorage/src/commonMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt
26. browserStorage/src/jsMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt
27. browserStorage/src/wasmJsMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt
28. browserStorage/worker/package.json
29. browserStorage/worker/worker.js
30. build.gradle.kts
31. desktopApp/build.gradle.kts
32. desktopApp/src/main/kotlin/com/example/dictionary/main.kt
33. desktopApp/src/test/kotlin/com/example/dictionary/NavigationCompatibilityTest.kt
34. gradle.properties
35. gradle/libs.versions.toml
36. iosApp/Configuration/Config.xcconfig

37. iosApp/iosApp/ContentView.swift
38. iosApp/iosApp/Info.plist
39. iosApp/iosApp/iOSApp.swift
40. settings.gradle.kts
41. shared/build.gradle.kts
42. shared/src/androidMain/kotlin/com/example/dictionary/data/AppStorage.kt
43. shared/src/androidMain/kotlin/com/example/dictionary/data/HttpClientFactory.android.kt
44. shared/src/iosMain/kotlin/com/example/dictionary/MainViewController.kt
45. shared/src/iosMain/kotlin/com/example/dictionary/data/AppStorage.kt
46. shared/src/iosMain/kotlin/com/example/dictionary/data/HttpClientFactory.ios.kt
47. shared/src/jsMain/kotlin/com/example/dictionary/data/AppStorage.kt
48. shared/src/jsMain/kotlin/com/example/dictionary/data/HttpClientFactory.js.kt
49. shared/src/jvmMain/kotlin/com/example/dictionary/data/AppStorage.kt
50. shared/src/jvmMain/kotlin/com/example/dictionary/data/HttpClientFactory.jvm.kt
51. shared/src/jvmTest/kotlin/com/example/dictionary/PersistenceTest.kt
52. shared/src/wasmJsMain/kotlin/com/example/dictionary/data/AppStorage.kt
53. shared/src/wasmJsMain/kotlin/com/example/dictionary/data/HttpClientFactory.wasmJs.kt
54. webApp/build.gradle.kts
55. webApp/src/webMain/kotlin/com/example/dictionary/main.kt
56. webApp/src/webMain/resources/index.html
57. webApp/src/webMain/resources/styles.css
58. webApp/webpack.config.d/storage.js

1. Models.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/Models.kt

The app’s containers for definitions, saved-history display items and complete lookup answers.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.serialization.Serializable
```

Makes Serializable available here from kotlinx.serialization.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 6

```
data class DictionaryEntry(
```

Defines DictionaryEntry as a data container.

Line 7

```
val word: String,
```

The word associated with this object.

Line 8

```
val source: String,
```

The dictionary that supplied the definition.

Line 9

```
val partOfSpeech: String = "",
```

A grammar label such as noun; empty when unavailable.

Line 10

```
val definition: String,
```

The meaning to display.

Line 11

```
val example: String = ""
```

An optional example sentence; empty when unavailable.

Line 12

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 13

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 14

```
data class HistoryItem(
```

Defines HistoryItem as a data container.

Line 15

```
val word: String,
```

The word associated with this object.

Line 16

```
val snippet: String,
```

A short preview of a saved definition for History.

Line 17

```
val formalEntries: List<DictionaryEntry>,
```

The list of formal dictionary definitions.

Line 18

```
val urbanEntries: List<DictionaryEntry>,
```

The list of slang definitions.

Line 19

```
val isFavorite: Boolean = false
```

Whether this history word is starred; false is the default.

Line 20

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
// A named result is easier to understand than
Triple.first/second/third.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 23

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 24

```
data class DictionaryResult(
```

Defines DictionaryResult as a data container.

Line 25

```
val formalEntries: List<DictionaryEntry>,
```

The list of formal dictionary definitions.

Line 26

```
val urbanEntries: List<DictionaryEntry>,
```

The list of slang definitions.

Line 27

```
val didYouMeanSuggestion: String? = null,
```

An optional corrected spelling; null means there is no suggestion.

Line 28

```
val isCached: Boolean = false
```

Whether this returned result came from the saved cache; false is the default.

Line 29

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

2. UiState.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/presentation/UiState.kt

All possible result-screen conditions in one sealed family.

Line 1

```
package com.example.dictionary.presentation
```

Places this file in the com.example.dictionary.presentation namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.domain.DictionaryEntry
```

Makes DictionaryEntry available here from com.example.dictionary.domain.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
sealed class UiState {
```

Limits result-screen states to the classes declared in this family.

Line 6

```
object Idle : UiState()
```

The user has not selected an active result yet.

Line 7

```
object Loading : UiState()
```

The app is currently waiting for a lookup.

Line 8

```
data class Success(
```

A successful state carries formal entries, slang entries and an optional suggestion.

Line 9

```
val formalEntries: List<DictionaryEntry>,
```

The list of formal dictionary definitions.

Line 10

```
val urbanEntries: List<DictionaryEntry>,
```

The list of slang definitions.

Line 11

```
val didYouMeanSuggestion: String? = null
```

An optional corrected spelling; null means there is no suggestion.

Line 12

```
) : UiState()
```

Finishes the Success data-class constructor and makes Success one of the permitted UiState cases.

Line 13

```
data class Error(val message: String) : UiState()
```

An error state carries the message that the screen should display.

Line 14

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

3. DictionaryRepository.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/DictionaryRepository.kt

The contract for lookups, observable history and favorite changes. It does not make HTTP calls itself.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
// Module 6: presentation depends on an interface, not
Ktor or Room.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 6

```
interface DictionaryRepository {
```

Defines DictionaryRepository as an interface of promised operations.

Line 7

```
suspend fun fetchDefinitions(word: String, forceRefresh:
Boolean = false): DictionaryResult?
```

Defines fetchDefinitions(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 8

```
fun observeHistory(): Flow<List<HistoryItem>>
```

Defines observeHistory(). Inputs are inside parentheses; the return type describes its answer.

Line 9

```
suspend fun toggleFavorite(word: String)
```

Defines toggleFavorite(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

4. SearchWordUseCase.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/SearchWordUseCase.kt

One small rule: normalize the query, then request its definitions.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
class SearchWordUseCase(private val repository:
DictionaryRepository) {
```

Receives the repository interface instead of constructing a data source itself.

Line 4

```
suspend operator fun invoke(word: String, forceRefresh:
Boolean = false): DictionaryResult? {
```

Allows an instance to be called like searchWordUseCase(word). The function can pause and accepts an optional forceRefresh flag.

Line 5

```
return
repository.fetchDefinitions(word.trim().lowercase(),
forceRefresh)
```

Cleans the input, asks the repository to fetch it, and sends its answer back unchanged.

Line 6

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

5. DictionaryViewModel.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/presentation/DictionaryViewModel.kt

Controls screen state and asynchronous actions. It observes persisted data and publishes state/events.

Line 1

```
package com.example.dictionary.presentation
```

Places this file in the com.example.dictionary.presentation namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.domain.DictionaryResult
```

Makes DictionaryResult available here from com.example.dictionary.domain.

Line 4

```
import com.example.dictionary.domain.HistoryItem
```

Makes HistoryItem available here from com.example.dictionary.domain.

Line 5

```
import com.example.dictionary.domain.HistoryUseCase
```

Makes HistoryUseCase available here from com.example.dictionary.domain.

Line 6

```
import com.example.dictionary.domain.SearchWordUseCase
```

Makes SearchWordUseCase available here from com.example.dictionary.domain.

Line 7

```
import
com.example.dictionary.domain.SlangPreferenceUseCase
```

Makes SlangPreferenceUseCase available here from com.example.dictionary.domain.

Line 8

```
import kotlinx.coroutines.CancellationException
```

Makes CancellationException available here from kotlinx.coroutines.

Line 9

```
import kotlinx.coroutines.CoroutineScope
```

Makes CoroutineScope available here from kotlinx.coroutines.

Line 10

```
import kotlinx.coroutines.Job
```

Makes Job available here from kotlinx.coroutines.

Line 11

```
import kotlinx.coroutines.flow.MutableSharedFlow
```

Makes MutableSharedFlow available here from kotlinx.coroutines.flow.

Line 12

```
import kotlinx.coroutines.flow.MutableStateFlow
```

Makes MutableStateFlow available here from kotlinx.coroutines.flow.

Line 13

```
import kotlinx.coroutines.flow.SharedFlow
```

Makes SharedFlow available here from kotlinx.coroutines.flow.

Line 14

```
import kotlinx.coroutines.flow.StateFlow
```

Makes StateFlow available here from kotlinx.coroutines.flow.

Line 15

```
import kotlinx.coroutines.launch
```

Makes launch available here from kotlinx.coroutines.

Line 16

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 17

```
// Module 7: this state holder uses plain Kotlin and Flow
 on every platform.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 18

```
class DictionaryViewModel(
```

Defines the shared screen-state holder. Its dependencies are passed into the constructor below.

Line 19

```
private val searchWordUseCase: SearchWordUseCase,
```

Receives/keeps searchWordUseCase as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 20

```
private val historyUseCase: HistoryUseCase,
```

Receives/keeps historyUseCase as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 21

```
private val slangPreferenceUseCase:
SlangPreferenceUseCase,
```

Receives/keeps slangPreferenceUseCase as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 22

```
private val scope: CoroutineScope
```

Receives/keeps scope as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 23

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 24

```
private var searchJob: Job? = null
```

Keeps the current search coroutine so a newer search can cancel it.

Line 25

```
private val _uiState =
MutableStateFlow<UiState>(UiState.Idle)
```

Owns the writable result state. Idle is its first value.

Line 26

```
val uiState: StateFlow<UiState> = _uiState
```

Exposes the current result state for observation; callers are not given the MutableStateFlow API.

Line 27

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 28

```
private val _searchHistory =
MutableStateFlow<List<HistoryItem>>(emptyList())
```

Starts the observable history as an empty list until Room sends saved rows.

Line 29

```
val searchHistory: StateFlow<List<HistoryItem>> =
_searchHistory
```

Exposes the history stream to the UI as read-only StateFlow.

Line 30

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 31

```
private val _showSlang = MutableStateFlow(false)
```

Starts the local preference state as false until its saved value arrives.

Line 32

```
val showSlang: StateFlow<Boolean> = _showSlang
```

Exposes the current slang preference to the UI.

Line 33

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 34

```
// Module 7: messages happen once; screen values belong
in StateFlow.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 35

```
private val _events = MutableSharedFlow<String>()
```

Creates the writable event stream with no replay buffer. Used for one-time messages.

Line 36

```
val events: SharedFlow<String> = _events
```

Exposes those events for collection by App's snackbar handler.

Line 37

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 38

```
init {
```

Starts the following observers when this ViewModel object is created.

Line 39

```
scope.launch {
```

Starts a coroutine inside the provided scope. It can wait for network/storage work without blocking the calling UI code.

Line 40

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 41

```
historyUseCase.observeHistory().collect {
_searchHistory.value = it }
```

Keeps listening to history changes and assigns each new list to the observable UI value.

Line 42

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 43

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 44

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 45

```
_events.emit("Could not read saved history.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 46

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 47

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 48

```
scope.launch {
```

Starts a coroutine inside the provided scope. It can wait for network/storage work without blocking the calling UI code.

Line 49

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 50

```
slangPreferenceUseCase.observeShowSlang().collect {
_showSlang.value = it }
```

Keeps listening to DataStore’s Boolean setting and updates the exposed state.

Line 51

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 52

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 53

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 54

```
_events.emit("Could not read your saved preference.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 55

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 56

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 57

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 58

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 59

```
fun searchWord(word: String, forceRefresh: Boolean =
false) {
```

Starts a word lookup; forceRefresh defaults to false so saved data can be used.

Line 60

```
if (word.isBlank()) return
```

Ignores empty/space-only input before starting any search work.

Line 61

```
searchJob?.cancel()
```

Cancels the previous search if one exists, so its late answer cannot replace the new search.

Line 62

```
searchJob = scope.launch {
```

Starts the new asynchronous search and retains its Job handle.

Line 63

```
val previousState = _uiState.value
```

Remembers the displayed state in case an explicit refresh fails.

Line 64

```
_uiState.value = UiState.Loading
```

Signals the screen to show its loading indicator.

Line 65

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 66

```
val result = searchWordUseCase(word, forceRefresh)
```

Calls the domain operation and waits without blocking for its result.

Line 67

```
if (result == null) {
```

The repository could not obtain a usable response; select the error/fallback behavior.

Line 68

```
if (forceRefresh && previousState is UiState.Success) {
```

When refreshing an already successful result, keep that result on failure.

Line 69

```
_uiState.value = previousState
```

Restores the earlier successful result instead of replacing it with an error.

Line 70

```
_events.emit("Could not refresh. Showing the saved
definition.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 71

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 72

```
_uiState.value = UiState.Error("Could not search. Check
your connection and try again.")
```

Publishes this readable error message as the current result-screen state. Compose observes the new value and draws its Error branch.

Line 73

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 74

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 75

```
showResult(word, result)
```

Turns this result into Success or a no-definitions Error.

Line 76

```
// Module 9: display cached data first, then refresh in
the background.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 77

```
if (result.isCached) {
```

A saved result was just displayed; now try to update it from the network.

Line 78

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 79

```
val fresh = searchWordUseCase(word, forceRefresh = true)
```

Bypasses the saved-word lookup for this background refresh request.

Line 80

```
if (fresh != null && hasDefinitions(fresh)) {
```

Only replace the displayed cached data when fresh definitions actually exist.

Line 81

```
showResult(word, fresh)
```

Publishes fresh definitions after the saved definitions were already usable.

Line 82

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 83

```
_events.emit("Showing saved definitions. A refresh is
unavailable.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 84

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 85

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 86

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 87

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 88

```
_events.emit("Showing saved definitions. A refresh is
unavailable.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 89

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 90

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 91

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 92

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 93

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 94

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 95

```
_uiState.value = UiState.Error("Could not load or save
this word. Please try again.")
```

Publishes this readable error message as the current result-screen state. Compose observes the new value and draws its Error branch.

Line 96

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 97

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 98

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 99

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 100

```
private fun hasDefinitions(result: DictionaryResult):
Boolean {
```

Small helper that asks whether either source supplied definitions.

Line 101

```
return result.formalEntries.isNotEmpty() ||
result.urbanEntries.isNotEmpty()
```

True when at least one dictionary list contains entries.

Line 102

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 103

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 104

```
private fun showResult(word: String, result:
DictionaryResult) {
```

Converts the domain answer into the state the result screen understands.

Line 105

```
_uiState.value = if (!hasDefinitions(result) &&
result.didYouMeanSuggestion == null) {
```

Chooses Error for no entries/no suggestion, otherwise Success.

Line 106

```
UiState.Error("No definitions found for
'${word.trim()}'.")
```

Creates the readable no-definitions state for this word.

Line 107

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 108

```
UiState.Success(result.formalEntries,
result.urbanEntries, result.didYouMeanSuggestion)
```

Publishes the two definition lists and optional spelling suggestion together.

Line 109

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 110

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 111

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 112

```
fun toggleFavorite(word: String) {
```

Receives a star-button action from App.

Line 113

```
scope.launch {
```

Starts a coroutine inside the provided scope. It can wait for network/storage work without blocking the calling UI code.

Line 114

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 115

```
historyUseCase.toggleFavorite(word)
```

Requests a persistent favorite update. History will redraw when the database Flow emits.

Line 116

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 117

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 118

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 119

```
_events.emit("Could not save this favorite.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 120

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 121

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 123

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 124

```
fun saveShowSlang(enabled: Boolean) {
```

Receives the new checkbox value from App.

Line 125

```
scope.launch {
```

Starts a coroutine inside the provided scope. It can wait for network/storage work without blocking the calling UI code.

Line 126

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 127

```
slangPreferenceUseCase.saveShowSlang(enabled)
```

Saves the Boolean through the preferences use case.

Line 128

```
_events.emit("Preference saved.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 129

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 130

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 131

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 132

```
_events.emit("Could not save your preference.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 133

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 134

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 135

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 136

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

6. DictionaryRepository.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/DictionaryRepository.kt

Checks Room before network requests, converts responses, saves useful definitions and exposes history.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.domain.DictionaryEntry
```

Makes DictionaryEntry available here from com.example.dictionary.domain.

Line 4

```
import com.example.dictionary.domain.DictionaryResult
```

Makes DictionaryResult available here from com.example.dictionary.domain.

Line 5

```
import com.example.dictionary.domain.DictionaryRepository
```

Makes DictionaryRepository available here from com.example.dictionary.domain.

Line 6

```
import com.example.dictionary.domain.HistoryItem
```

Makes HistoryItem available here from com.example.dictionary.domain.

Line 7

```
import com.example.dictionary.data.local.HistoryDao
```

Makes HistoryDao available here from com.example.dictionary.data.local.

Line 8

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 9

```
import kotlinx.coroutines.flow.map
```

Makes map available here from kotlinx.coroutines.flow.

Line 10

```
import kotlinx.serialization.encodeToString
```

Makes encodeToString available here from kotlinx.serialization.

Line 11

```
import kotlinx.coroutines.CancellationException
```

Makes CancellationException available here from kotlinx.coroutines.

Line 12

```
import io.ktor.client.*
```

Makes * available from io.ktor.client. A star imports accessible declarations from that package.

Line 13

```
import io.ktor.client.call.*
```

Makes * available from io.ktor.client.call. A star imports accessible declarations from that package.

Line 14

```
import io.ktor.client.request.*
```

Makes * available from io.ktor.client.request. A star imports accessible declarations from that package.

Line 15

```
import io.ktor.http.*
```

Makes * available from io.ktor.http. A star imports accessible declarations from that package.

Line 16

```
import kotlinx.serialization.Serializable
```

Makes Serializable available here from kotlinx.serialization.

Line 17

```
import kotlinx.serialization.json.*
```

Makes * available from kotlinx.serialization.json. A star imports accessible declarations from that package.

Line 18

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 19

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 20

```
private data class UrbanResponse(val list:
List<UrbanDefinition>)
```

Models the top-level response containing a list of slang definitions.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 23

```
private data class UrbanDefinition(
```

Models a slang entry’s fields so JSON can be decoded into Kotlin values.

Line 24

```
val definition: String,
```

The meaning to display.

Line 25

```
val word: String,
```

The word associated with this object.

Line 26

```
val example: String = ""
```

An optional example sentence; empty when unavailable.

Line 27

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 28

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 29

```
// Module 6: App supplies the client and API key through
the constructor.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 30

```
class DictionaryRepositoryImpl(
```

Implements the domain repository promise using injected Room operations and Ktor.

Line 31

```
private val dao: HistoryDao,
```

Receives/keeps dao as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 32

```
private val client: HttpClient,
```

Receives/keeps client as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 33

```
private val apiKey: String
```

Receives/keeps apiKey as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 34

```
) : DictionaryRepository {
```

Finishes the injected constructor and declares that this class implements the domain DictionaryRepository interface.

Line 35

```
private val json = Json { ignoreUnknownKeys = true }
```

Creates a reusable JSON converter that ignores fields the app does not model.

Line 36

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 37

```
override suspend fun fetchDefinitions(word: String,
forceRefresh: Boolean): DictionaryResult? {
```

Implements a possibly-pausing dictionary lookup. A null answer represents an unusable failed lookup.

Line 38

```
val sanitizedInput = word.trim().lowercase()
```

Removes surrounding spaces and converts letters to lowercase for consistent lookups/keys.

Line 39

```
// Module 9: read the local database first, unless
explicitly refreshing.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 40

```
if (!forceRefresh) {
```

Use saved data only when the caller has not explicitly requested a refresh.

Line 41

```
val cached = dao.findWord(sanitizedInput)
```

Reads the saved row for this normalized word.

Line 42

```
if (cached != null) {
```

There is an existing row, so return its definitions before making online requests.

Line 43

```
return json.decodeFromString<DictionaryResult>(cached.res
ultJson).copy(isCached = true)
```

Converts saved JSON back to DictionaryResult and marks this returned copy as cached.

Line 44

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 45

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
var failedSources = 0
```

Counts dictionary sources that failed during this network lookup.

Line 47

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 48

```
val urbanEntries = try {
```

Attempts Urban Dictionary independently so one service’s failure need not discard the other service’s answer.

Line 49

```
val httpResponse =
client.get("https://api.urbandictionary.com/v0/define") {
```

Starts the Urban Dictionary GET request; the following parameter contains the word.

Line 50

```
parameter("term", sanitizedInput)
```

Adds the word as the term query parameter. Ktor handles query-parameter encoding.

Line 51

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 52

```
if (httpResponse.status != HttpStatusCode.OK) {
```

Checks the HTTP response status rather than assuming every response contains valid definitions.

Line 53

```
error("Urban Dictionary is unavailable.")
```

Raises a failure that the enclosing catch handles as a failed dictionary source.

Line 54

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 55

```
val response: UrbanResponse = httpResponse.body()
```

Uses Ktor’s JSON converter to read the response into the typed slang response model.

Line 56

```
response.list.take(3).map {
```

Keeps up to three slang entries and converts each API model into the app’s DictionaryEntry model.

Line 57

```
DictionaryEntry(
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 58

```
word = it.word,
```

Copies the word supplied by the slang entry.

Line 59

```
source = "Urban Dictionary",
```

Labels the entry with the dictionary that supplied it.

Line 60

```
definition = it.definition.replace("[", "").replace("]",
"),
```

Removes Urban Dictionary’s square-bracket link markers from the displayed definition.

Line 61

```
example = it.example.replace("[", "").replace("]", "")
```

Removes those markers from the example sentence too.

Line 62

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 63

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 64

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 65

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 66

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 67

```
failedSources++
```

Adds one to the number of failed sources.

Line 68

```
println("Dictionary request failed:
${e::class.simpleName}")
```

Prints the error's class name for diagnosis without printing a request URL/API key.

Line 69

```
emptyList()
```

Uses an empty list as this block's answer; callers can safely handle no entries.

Line 70

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 71

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 72

```
var mwDidYouMean: String? = null
```

Starts with no spelling correction; the formal API may supply one.

Line 73

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 74

```
val formalEntries = try {
```

Attempts the formal dictionary request after the slang request.

Line 75

```
val httpResponse = client.get(
```

Stores a value/reference named httpResponse from the expression on the right.

Line 76

```
"https://dictionaryapi.com/api/v3/references/collegiate/j
son/${sanitizedInput.encodeURLPathPart()}"
```

Builds the formal dictionary URL and encodes the word for its URL path.

Line 77

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 78

```
parameter("key", apiKey)
```

Adds the repository's injected API key to the formal dictionary request.

Line 79

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 80

```
if (httpResponse.status != HttpStatusCode.OK) {
```

Checks the HTTP response status rather than assuming every response contains valid definitions.

Line 81

```
error("Merriam-Webster is unavailable.")
```

Raises a failure that the enclosing catch handles as a failed dictionary source.

Line 82

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 83

```
// This API returns either spelling strings OR definition
 objects.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 84

```
// Only inspect the first item; serialization handles the
 definition fields.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 85

```
val response: JsonArray = httpResponse.body()
```

Reads the formal response as an array because this API returns either objects or spelling strings.

Line 86

```
if (response.firstOrNull() is JsonPrimitive) {
```

Distinguishes a suggestion-string response from a definition-object response.

Line 87

```
mwDidYouMean = response.first().jsonPrimitive.content
```

Keeps the first suggested word for the clickable correction banner.

Line 88

```
emptyList()
```

Uses an empty list as this block’s answer; callers can safely handle no entries.

Line 89

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 90

```
val definitions = json.decodeFromJsonElement<List<FormalD
efinition>>(response)
```

Uses typed serialization for the formal objects after checking which array format was returned.

Line 91

```
val entries = mutableListOf<DictionaryEntry>()
```

Creates a temporary list to collect the formal definitions converted into app models.

Line 92

```
for (item in definitions.take(3)) {
```

Processes up to three formal dictionary items.

Line 93

```
if (item.shortdef.isNotEmpty()) {
```

Skips formal items that do not have a short definition.

Line 94

```
entries.add(DictionaryEntry(
```

Adds one converted formal definition to the temporary result list.

Line 95

```
word = sanitizedInput,
```

Uses the normalized search term for this formal entry.

Line 96

```
source = "Merriam-Webster",
```

Labels the entry with the dictionary that supplied it.

Line 97

```
partOfSpeech = item.fl,
```

Copies the API’s fl field, such as noun, into the app’s partOfSpeech property.

Line 98

```
definition = item.shortdef.first()
```

Uses the first short definition from this formal item.

Line 99

```
))
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 100

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 101

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 102

```
entries
```

The last expression of this try branch is its result: return the accumulated formal-entry list as formalEntries.

Line 103

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 104

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 105

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 106

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 107

```
failedSources++
```

Adds one to the number of failed sources.

Line 108

```
println("Dictionary request failed:
${e::class.simpleName}")
```

Prints the error's class name for diagnosis without printing a request URL/API key.

Line 109

```
emptyList()
```

Uses an empty list as this block's answer; callers can safely handle no entries.

Line 110

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 111

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 112

```
if (formalEntries.isEmpty() && urbanEntries.isEmpty() &&
failedSources > 0 && mwDidYouMean == null) {
```

Keeps the first suggested word for the clickable correction banner.

Line 113

```
return null
```

Sends no usable result to the caller. The ViewModel decides the error screen.

Line 114

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 115

```
val result = DictionaryResult(formalEntries,
urbanEntries, mwDidYouMean)
```

Packages formal entries, slang entries and the optional correction into one named answer.

Line 116

```
// Save useful results, including when only one
dictionary is reachable.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 117

```
if (formalEntries.isNotEmpty() ||
urbanEntries.isNotEmpty()) {
```

Save only actual definitions; a spelling suggestion alone is not a saved dictionary result.

Line 118

```
val snippet = formalEntries.firstOrNull()?.definition
```

Prefers a formal definition for the history preview; the next line falls back to slang.

Line 119

```
?: urbanEntries.first().definition
```

Uses the first slang definition if there is no formal one. The earlier condition ensures one list has data.

Line 120

```
dao.saveWord(sanitizedInput, snippet,
json.encodeToString(result))
```

Serializes the result lists into JSON and stores the word/preview/result through Room.

Line 121

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
return result
```

Returns the lookup answer to the use case, then the ViewModel.

Line 123

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 124

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 125

```
// Modules 7 and 9: Room emits history again whenever
saved data changes.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 126

```
override fun observeHistory(): Flow<List<HistoryItem>> {
```

Exposes saved history as a stream, not just one list read once.

Line 127

```
return dao.observeHistory().map { savedWords ->
```

Transforms each new list emitted by the Room query into UI-friendly history items.

Line 128

```
savedWords.map { saved ->
```

Converts every saved row in the latest emitted list.

Line 129

```
val result =
json.decodeFromString<DictionaryResult>(saved.resultJson)
```

Reconstructs the definition lists from the row's saved JSON text.

Line 130

```
HistoryItem(saved.word, saved.snippet,
result.formalEntries,
```

Starts the history item with the word, preview and formal definitions.

Line 131

```
result.urbanEntries, saved.isFavorite)
```

Includes saved slang definitions and the current database favorite flag.

Line 132

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 133

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 134

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 135

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 136

```
override suspend fun toggleFavorite(word: String) {
```

Implements the persistent star/unstar operation promised by the interface.

Line 137

```
dao.toggleFavorite(word.trim().lowercase())
```

Normalizes the word and forwards the update to Room.

Line 138

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 139

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 140

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 141

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 142

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 143

```
private data class FormalDefinition(
```

Models only the formal API fields used by this app; other fields are ignored.

Line 144

```
val fl: String = "",
```

The API field fl contains a part of speech; missing values become an empty String.

Line 145

```
val shortdef: List<String> = emptyList()
```

The formal API’s list of short definitions; missing values become an empty list.

Line 146

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

7. App.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/App.kt

Connects dependencies, observes state, and opens the correct screen. It delegates networking/storage work.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.foundation.BorderStroke
```

Makes BorderStroke available here from androidx.compose.foundation.

Line 4

```
import androidx.compose.foundation.layout.*
```

Makes * available from androidx.compose.foundation.layout. A star imports accessible declarations from that package.

Line 5

```
import androidx.compose.material3.*
```

Makes * available from androidx.compose.material3. A star imports accessible declarations from that package.

Line 6

```
import androidx.compose.runtime.*
```

Makes * available from androidx.compose.runtime. A star imports accessible declarations from that package.

Line 7

```
import androidx.compose.ui.Modifier
```

Makes Modifier available here from androidx.compose.ui.

Line 8

```
import androidx.compose.ui.graphics.Color
```

Makes Color available here from androidx.compose.ui.graphics.

Line 9

```
import androidx.compose.ui.text.font.FontWeight
```

Makes FontWeight available here from androidx.compose.ui.text.font.

Line 10

```
import androidx.compose.ui.unit.dp
```

Makes dp available here from androidx.compose.ui.unit.

Line 11

```
import androidx.savedstate.read
```

Makes read available here from androidx.savedstate.

Line 12

```
import androidx.navigation.NavType
```

Makes NavType available here from androidx.navigation.

Line 13

```
import androidx.navigation.navArgument
```

Makes navArgument available here from androidx.navigation.

Line 14

```
import androidx.navigation.navDeepLink
```

Makes navDeepLink available here from androidx.navigation.

Line 15

```
import androidx.navigation.compose.NavHost
```

Makes NavHost available here from androidx.navigation.compose.

Line 16

```
import androidx.navigation.compose.composable
```

Makes composable available here from androidx.navigation.compose.

Line 17

```
import
androidx.navigation.compose.currentBackStackEntryAsState
```

Makes currentBackStackEntryAsState available here from androidx.navigation.compose.

Line 18

```
import androidx.navigation.compose.rememberNavController
```

Makes rememberNavController available here from androidx.navigation.compose.

Line 19

```
import com.example.dictionary.data.AppStorage
```

Makes AppStorage available here from com.example.dictionary.data.

Line 20

```
import com.example.dictionary.data.DictionaryPreferences
```

Makes DictionaryPreferences available here from com.example.dictionary.data.

Line 21

```
import
com.example.dictionary.data.DictionaryRepositoryImpl
```

Makes DictionaryRepositoryImpl available here from com.example.dictionary.data.

Line 22

```
import com.example.dictionary.data.createHttpClient
```

Makes createHttpClient available here from com.example.dictionary.data.

Line 23

```
import com.example.dictionary.domain.HistoryUseCase
```

Makes HistoryUseCase available here from com.example.dictionary.domain.

Line 24

```
import com.example.dictionary.domain.SearchWordUseCase
```

Makes SearchWordUseCase available here from com.example.dictionary.domain.

Line 25

```
import
com.example.dictionary.domain.SlangPreferenceUseCase
```

Makes SlangPreferenceUseCase available here from com.example.dictionary.domain.

Line 26

```
import
com.example.dictionary.presentation.DictionaryViewModel
```

Makes DictionaryViewModel available here from com.example.dictionary.presentation.

Line 27

```
import com.example.dictionary.presentation.UiState
```

Makes UiState available here from com.example.dictionary.presentation.

Line 28

```
import com.example.dictionary.ui.HistoryScreen
```

Makes HistoryScreen available here from com.example.dictionary.ui.

Line 29

```
import com.example.dictionary.ui.InputScreen
```

Makes InputScreen available here from com.example.dictionary.ui.

Line 30

```
import com.example.dictionary.ui.ResultScreen
```

Makes ResultScreen available here from com.example.dictionary.ui.

Line 31

```
import io.ktor.http.encodeURLParameter
```

Makes encodeURLParameter available here from io.ktor.http.

Line 32

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 33

```
// Module 8: a small, fixed set of destinations, with a
word for the result.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 34

```
sealed class Destination {
```

Defines the small set of places the app can open. Result also carries a word.

Line 35

```
object Input : Destination()
```

Declares this fixed state/destination or the values carried by this case.

Line 36

```
object History : Destination()
```

Declares this fixed state/destination or the values carried by this case.

Line 37

```
data class Result(val word: String) : Destination()
```

Defines Result as a data container.

Line 38

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 39

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 40

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 41

```
fun App(storage: AppStorage) {
```

Defines App(). Inputs are inside parentheses; the return type describes its answer.

Line 42

```
val scope = rememberCoroutineScope()
```

Creates the coroutine scope used by this App. Compose cancels its jobs when this App leaves composition.

Line 43

```
val client = remember { createHttpClient() }
```

Creates the shared HTTP client once for this composition location, rather than once per redraw.

Line 44

```
DisposableEffect(client) {
```

Registers cleanup for this particular HTTP client.

Line 45

```
onDispose { client.close() }
```

Closes network resources when this effect is removed or its client changes.

Line 46

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 47

```
// Module 6: simple constructor injection; no DI
framework is needed.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 48

```
val viewModel = remember(storage) {
```

Keeps the state holder while this storage object stays the same. This is memory retention, not disk storage.

Line 49

```
val repository = DictionaryRepositoryImpl(
```

Builds the real repository implementation. The following arguments supply its dependencies.

Line 50

```
storage.database.historyDao(), client, "<existing API
key>"
```

Supplies database operations, the HTTP client and the existing API key. The guide hides the literal key only; the source is unchanged.

Line 51

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 52

```
DictionaryViewModel(
```

Builds the shared state holder using the three use cases and coroutine scope below.

Line 53

```
SearchWordUseCase(repository),
```

Gives the ViewModel the operation that searches through this repository.

Line 54

```
HistoryUseCase(repository),
```

Gives it history/favorite operations using the same repository.

Line 55

```
SlangPreferenceUseCase(DictionaryPreferences(storage.pref
erences)),
```

Wraps the saved preferences in their implementation and use case.

Line 56

```
scope
```

Supplies the Compose-owned coroutine scope as the final ViewModel constructor argument.

Line 57

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 58

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 59

```
val navController = rememberNavController()
```

Creates and remembers the object that tracks navigation destinations and their back stack.

Line 60

```
val backStackEntry by
navController.currentBackStackEntryAsState()
```

Observes the current navigation entry so the selected tab updates when navigation changes.

Line 61

```
val currentRoute = backStackEntry?.destination?.route ?:
"input"
```

Reads the active route; uses input before an entry is available.

Line 62

```
var searchQuery by remember { mutableStateOf("") }
```

Keeps the text currently typed in Input in Compose memory.

Line 63

```
var lastWord by remember { mutableStateOf("") }
```

Remembers the last result word for the bottom Result button; the displayed result itself reads its route argument.

Line 64

```
val state by viewModel.uiState.collectAsState()
```

Observes the result state. A change causes the parts of the screen reading it to redraw.

Line 65

```
val history by viewModel.searchHistory.collectAsState()
```

Observes the saved history list exposed by the ViewModel.

Line 66

```
val showSlang by viewModel.showSlang.collectAsState()
```

Observes the saved slang-display default exposed by the ViewModel.

Line 67

```
val snackbarHostState = remember { SnackbarHostState() }
```

Keeps the object controlling the temporary message bar.

Line 68

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 69

```
// Module 7: collect one-time messages separately from
persistent screen state.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 70

```
LaunchedEffect(viewModel) {
```

Starts one event collector for this ViewModel; it is cancelled if the effect leaves composition.

Line 71

```
viewModel.events.collect { message ->
snackbarHostState.showSnackbar(message) }
```

Waits for one-time messages and shows each one in a snackbar.

Line 72

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 73

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 74

```
fun navigate(destination: Destination) {
```

Defines App’s helper for changing screens.

Line 75

```
val route = when (destination) {
```

Selects the route string based on the kind of destination.

Line 76

```
Destination.Input -> "input"
```

The Input destination uses the literal route input.

Line 77

```
Destination.History -> "history"
```

The History destination uses the literal route history.

Line 78

```
is Destination.Result -> {
```

For a result destination, its word must be included in the route.

Line 79

```
val word = destination.word.trim().lowercase()
```

Normalizes the word before putting it in a navigation route.

Line 80

```
if (word.isBlank()) "result" else
"result/${word.encodeURLParameter()}"
```

Uses an empty-result route for blank input; otherwise URL-encodes the word so route punctuation is handled safely.

Line 81

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 82

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 83

```
navController.navigate(route) {
```

Asks the controller to open the chosen route.

Line 84

```
launchSingleTop = true
```

Avoids adding another copy when this destination is already at the top. It does not remove older different entries.

Line 85

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 86

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 87

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 88

```
MaterialTheme(
```

Supplies the colors/styles that Material components read while drawing.

Line 89

```
colorScheme = lightColorScheme(
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 90

```
primary = Color.Black,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 91

```
onPrimary = Color.White,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 92

```
surface = Color.White,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 93

```
onSurface = Color.Black,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 94

```
surfaceVariant = Color(0xFFF4F4F6)
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 95

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 96

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 97

```
Scaffold(
```

Creates the page frame, including the message area, bottom navigation and screen content.

Line 98

```
snackbarHost = { SnackbarHost(snackbarHostState) },
```

Places the message bar connected to snackbarHostState in the Scaffold.

Line 99

```
bottomBar = {
```

Starts the content shown at the bottom of the app.

Line 100

```
Surface(
```

A visual container providing background, border and shape.

Line 101

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 102

```
color = Color.White,
```

Sets this element’s color.

Line 103

```
border = BorderStroke(1.dp, Color.Black)
```

Sets the outline width and color.

Line 104

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 105

```
NavigationBar(containerColor = Color.White,
tonalElevation = 0.dp) {
```

Draws the bottom-navigation container.

Line 106

```
NavigationBarItem(
```

Draws one bottom-navigation choice.

Line 107

```
icon = { Text("01", fontWeight = FontWeight.Bold) },
```

Defines the icon content; here the item can use text instead of an image.

Line 108

```
label = { Text("INPUT") },
```

Defines the label content of a navigation item.

Line 109

```
selected = currentRoute == "input",
```

Marks this bottom item selected when its route matches the current screen.

Line 110

```
onClick = { navigate(Destination.Input) }
```

Makes tapping this bottom item open its associated destination.

Line 111

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 112

```
NavigationBarItem(
```

Draws one bottom-navigation choice.

Line 113

```
icon = { Text("02", fontWeight = FontWeight.Bold) },
```

Defines the icon content; here the item can use text instead of an image.

Line 114

```
label = { Text("RESULT") },
```

Defines the label content of a navigation item.

Line 115

```
selected = currentRoute.startsWith("result"),
```

Marks this bottom item selected when its route matches the current screen.

Line 116

```
onClick = { navigate(Destination.Result(lastWord)) }
```

Makes tapping this bottom item open its associated destination.

Line 117

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 118

```
NavigationBarItem(
```

Draws one bottom-navigation choice.

Line 119

```
icon = { Text("03", fontWeight = FontWeight.Bold) },
```

Defines the icon content; here the item can use text instead of an image.

Line 120

```
label = { Text("HISTORY") },
```

Defines the label content of a navigation item.

Line 121

```
selected = currentRoute == "history",
```

Marks this bottom item selected when its route matches the current screen.

Line 122

```
onClick = { navigate(Destination.History) }
```

Makes tapping this bottom item open its associated destination.

Line 123

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 124

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 125

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 126

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 127

```
) { innerPadding ->
```

Receives the padding needed to avoid drawing screen content under Scaffold bars.

Line 128

```
NavHost(
```

Connects a navigation controller to a group of screen definitions.

Line 129

```
navController = navController,
```

Tells NavHost which remembered navigation controller to use.

Line 130

```
startDestination = "input",
```

Opens Input when the graph first starts without another destination request.

Line 131

```
modifier = Modifier.fillMaxSize().padding(innerPadding)
```

Applies the following layout/appearance instructions to this UI element.

Line 132

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 133

```
composable("input") {
```

Defines the Input screen destination.

Line 134

```
InputScreen(
```

Calls the shared input UI with the supplied values and action callbacks.

Line 135

```
query = searchQuery,
```

Supplies the current text value; InputScreen does not own a second independent query.

Line 136

```
onQueryChange = { searchQuery = it },
```

When typing changes, store the new text in App’s searchQuery.

Line 137

```
onSearchClick = { navigate(Destination.Result(it)) },
```

When Input reports a search, navigate to that word’s result route.

Line 138

```
showSlang = showSlang,
```

Supplies the saved slang-default value to this part of the UI.

Line 139

```
onShowSlangChange = { viewModel.saveShowSlang(it) }
```

Forward the checkbox value to the ViewModel, which saves it through the use case.

Line 140

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 141

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 142

```
composable("result") {
```

Defines the result tab when no word has been selected.

Line 143

```
ResultScreen("", UiState.Idle, showSlang, {}, {})
```

Shows the initial result screen with no word and no-op action callbacks.

Line 144

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 145

```
// Module 8: pass the word through the route, including
deep links.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 146

```
composable(
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 147

```
route = "result/{word}",
```

Declares a route with a required word placeholder.

Line 148

```
arguments = listOf(navArgument("word") { type =
NavType.StringType }),
```

Specifies that the word route argument is a String.

Line 149

```
deepLinks = listOf(navDeepLink { uriPattern =
"dictionary://word/{word}" })
```

Associates this destination with dictionary://word/{word}. OS URL registration is a separate platform step.

Line 150

```
) { entry ->
```

Starts a branch/callback and gives the incoming value the name shown before the arrow.

Line 151

```
val word = entry.arguments?.read { getString("word")
}.orEmpty()
```

Reads the selected word using the SavedState API expected by the current Navigation library; null arguments become an empty string.

Line 152

```
LaunchedEffect(word) {
```

Runs the lookup effect when this destination enters composition, or when its word changes.

Line 153

```
lastWord = word
```

Updates the word used by the bottom Result tab.

Line 154

```
searchQuery = word
```

Also puts the selected word into Input’s text value.

Line 155

```
viewModel.searchWord(word)
```

Starts the result lookup through the state holder.

Line 156

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 157

```
ResultScreen(
```

Calls the shared result UI with its word, state and action callbacks.

Line 158

```
word = word,
```

Passes the relevant word to the object/component being built.

Line 159

```
state = state,
```

Passes the current sealed result state to the UI.

Line 160

```
showSlangByDefault = showSlang,
```

Supplies the saved slang-default value to this part of the UI.

Line 161

```
onRefresh = { viewModel.searchWord(word, forceRefresh =
true) },
```

Requests a new lookup that skips the initial saved-word lookup.

Line 162

```
onSearchSuggestion = { navigate(Destination.Result(it)) }
```

Tapping a suggestion navigates to that suggested word.

Line 163

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 164

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 165

```
composable("history") {
```

Defines the History screen destination.

Line 166

```
HistoryScreen(
```

Calls the shared saved-history UI with its list and callbacks.

Line 167

```
history = history,
```

Passes the observed history list into its screen.

Line 168

```
onSelectWord = { navigate(Destination.Result(it)) },
```

Opening a history row navigates to the selected word; normal lookup can return its saved definition.

Line 169

```
onToggleFavorite = { viewModel.toggleFavorite(it) }
```

Forward the star action to the ViewModel instead of modifying the row in the screen.

Line 170

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 171

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 172

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 173

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 174

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 175

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

8. InputScreen.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/ui/InputScreen.kt

Draws search input/settings and reports user actions via callbacks.

Line 1

```
package com.example.dictionary.ui
```

Places this file in the com.example.dictionary.ui namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.foundation.BorderStroke
```

Makes BorderStroke available here from androidx.compose.foundation.

Line 4

```
import androidx.compose.foundation.layout.*
```

Makes * available from androidx.compose.foundation.layout. A star imports accessible declarations from that package.

Line 5

```
import androidx.compose.material3.*
```

Makes * available from androidx.compose.material3. A star imports accessible declarations from that package.

Line 6

```
import androidx.compose.runtime.*
```

Makes * available from androidx.compose.runtime. A star imports accessible declarations from that package.

Line 7

```
import androidx.compose.ui.Alignment
```

Makes Alignment available here from androidx.compose.ui.

Line 8

```
import androidx.compose.ui.Modifier
```

Makes Modifier available here from androidx.compose.ui.

Line 9

```
import androidx.compose.ui.graphics.Color
```

Makes Color available here from androidx.compose.ui.graphics.

Line 10

```
import androidx.compose.ui.text.font.FontFamily
```

Makes FontFamily available here from androidx.compose.ui.text.font.

Line 11

```
import androidx.compose.ui.text.font.FontWeight
```

Makes FontWeight available here from androidx.compose.ui.text.font.

Line 12

```
import androidx.compose.ui.unit.dp
```

Makes dp available here from androidx.compose.ui.unit.

Line 13

```
import androidx.compose.ui.unit.sp
```

Makes sp available here from androidx.compose.ui.unit.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 16

```
fun InputScreen(
```

Defines InputScreen(). Inputs are inside parentheses; the return type describes its answer.

Line 17

```
query: String,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 18

```
onQueryChange: (String) -> Unit,
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 19

```
onSearchClick: (String) -> Unit,
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 20

```
showSlang: Boolean,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 21

```
onShowSlangChange: (Boolean) -> Unit
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 22

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 23

```
Surface(modifier = Modifier.fillMaxSize(), color =
Color(0xFFF9F9FB)) {
```

A visual container providing background, border and shape.

Line 24

```
Column(
```

A vertical layout: the children below are arranged top to bottom.

Line 25

```
modifier = Modifier.fillMaxSize().padding(32.dp),
```

Applies the following layout/appearance instructions to this UI element.

Line 26

```
horizontalAlignment = Alignment.Start,
```

Controls horizontal alignment of children.

Line 27

```
verticalArrangement = Arrangement.Center
```

Controls how children are spread vertically.

Line 28

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 29

```
Text(
```

Draws text using the supplied value/style.

Line 30

```
text = "// KMP Dictionary",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 31

```
style = MaterialTheme.typography.labelLarge.copy(
```

Chooses a theme text style or a customized copy of it.

Line 32

```
letterSpacing = 2.sp,
```

Sets the spacing between letters.

Line 33

```
fontWeight = FontWeight.Bold
```

Sets how bold the text is.

Line 34

```
),
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 35

```
color = Color.Black
```

Sets this element’s color.

Line 36

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 37

```
Spacer(modifier = Modifier.height(12.dp))
```

Adds empty space between UI elements.

Line 38

```
Text(
```

Draws text using the supplied value/style.

Line 39

```
text = "Look up definitions & slang.",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 40

```
style = MaterialTheme.typography.headlineLarge.copy(
```

Chooses a theme text style or a customized copy of it.

Line 41

```
fontWeight = FontWeight.ExtraBold,
```

Sets how bold the text is.

Line 42

```
fontSize = 32.sp
```

Sets text size using sp, which respects font scaling.

Line 43

```
),
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 44

```
color = Color.Black
```

Sets this element's color.

Line 45

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 47

```
Spacer(modifier = Modifier.height(40.dp))
```

Adds empty space between UI elements.

Line 48

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 49

```
OutlinedTextField(
```

Draws the editable word input with an outline.

Line 50

```
value = query,
```

Draw the current query supplied by App.

Line 51

```
onValueChange = onQueryChange,
```

Send newly typed text to App rather than keeping a separate local query.

Line 52

```
placeholder = { Text("enter target word...", color =
Color.Gray) },
```

Defines the hint shown while the text field is empty.

Line 53

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 54

```
singleLine = true,
```

Keeps the input to a single line.

Line 55

```
shape = Shapes().extraSmall,
```

Chooses the component’s corner shape.

Line 56

```
colors = OutlinedTextFieldDefaults.colors(
```

Supplies the Material component color configuration.

Line 57

```
focusedBorderColor = Color.Black,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 58

```
unfocusedBorderColor = Color.DarkGray,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 59

```
focusedContainerColor = Color.White,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 60

```
unfocusedContainerColor = Color.White
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 61

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 62

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 63

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 64

```
Row(verticalAlignment = Alignment.CenterVertically) {
```

A horizontal layout: the children below are arranged side by side.

Line 65

```
Checkbox(checked = showSlang, onCheckedChange =
onShowSlangChange)
```

Draw the saved slang-default value; send changes through the supplied preference callback.

Line 66

```
Text("Show slang by default (may contain explicit
language)",
```

Draws text using the supplied value/style.

Line 67

```
style = MaterialTheme.typography.bodySmall)
```

Chooses a theme text style or a customized copy of it.

Line 68

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 69

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 70

```
Spacer(modifier = Modifier.height(16.dp))
```

Adds empty space between UI elements.

Line 71

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 72

```
Button(
```

Draws a clickable button and connects the action supplied below.

Line 73

```
onClick = { if (query.isNotBlank()) onSearchClick(query)
},
```

When Search is tapped, ignore blank input and send the query to App’s callback.

Line 74

```
modifier = Modifier.fillMaxWidth().height(52.dp),
```

Applies the following layout/appearance instructions to this UI element.

Line 75

```
shape = Shapes().extraSmall,
```

Chooses the component’s corner shape.

Line 76

```
colors = ButtonDefaults.buttonColors(containerColor =
Color.Black)
```

Supplies the Material component color configuration.

Line 77

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 78

```
Text(
```

Draws text using the supplied value/style.

Line 79

```
text = "EXECUTE SEARCH →",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 80

```
style = MaterialTheme.typography.titleSmall.copy(
```

Chooses a theme text style or a customized copy of it.

Line 81

```
letterSpacing = 1.5.sp,
```

Sets the spacing between letters.

Line 82

```
fontWeight = FontWeight.Bold
```

Sets how bold the text is.

Line 83

```
),
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 84

```
color = Color.White
```

Sets this element’s color.

Line 85

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 86

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 87

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 88

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 89

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

9. ResultScreen.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/ui/ResultScreen.kt

Draws the result state and definition cards. Slang show/hide is local UI state.

Line 1

```
package com.example.dictionary.ui
```

Places this file in the com.example.dictionary.ui namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.foundation.BorderStroke
```

Makes BorderStroke available here from androidx.compose.foundation.

Line 4

```
import androidx.compose.foundation.background
```

Makes background available here from androidx.compose.foundation.

Line 5

```
import androidx.compose.foundation.clickable
```

Makes clickable available here from androidx.compose.foundation.

Line 6

```
import androidx.compose.foundation.layout.*
```

Makes * available from androidx.compose.foundation.layout. A star imports accessible declarations from that package.

Line 7

```
import androidx.compose.foundation.lazy.LazyColumn
```

Makes LazyColumn available here from androidx.compose.foundation.lazy.

Line 8

```
import androidx.compose.foundation.lazy.items
```

Makes items available here from androidx.compose.foundation.lazy.

Line 9

```
import androidx.compose.material3.*
```

Makes * available from androidx.compose.material3. A star imports accessible declarations from that package.

Line 10

```
import androidx.compose.runtime.*
```

Makes * available from androidx.compose.runtime. A star imports accessible declarations from that package.

Line 11

```
import androidx.compose.ui.Alignment
```

Makes Alignment available here from androidx.compose.ui.

Line 12

```
import androidx.compose.ui.Modifier
```

Makes Modifier available here from androidx.compose.ui.

Line 13

```
import androidx.compose.ui.graphics.Color
```

Makes Color available here from androidx.compose.ui.graphics.

Line 14

```
import androidx.compose.ui.text.font.FontWeight
```

Makes FontWeight available here from androidx.compose.ui.text.font.

Line 15

```
import androidx.compose.ui.unit.dp
```

Makes dp available here from androidx.compose.ui.unit.

Line 16

```
import androidx.compose.ui.unit.sp
```

Makes sp available here from androidx.compose.ui.unit.

Line 17

```
import com.example.dictionary.domain.DictionaryEntry
```

Makes DictionaryEntry available here from com.example.dictionary.domain.

Line 18

```
import com.example.dictionary.presentation.UiState
```

Makes UiState available here from com.example.dictionary.presentation.

Line 19

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 20

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 21

```
fun ResultScreen(
```

Defines ResultScreen(). Inputs are inside parentheses; the return type describes its answer.

Line 22

```
word: String,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 23

```
state: UiState,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 24

```
showSlangByDefault: Boolean,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 25

```
onRefresh: () -> Unit,
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 26

```
onSearchSuggestion: (String) -> Unit
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 27

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 28

```
var showUrbanSlang by remember(word, showSlangByDefault)
{ mutableStateOf(showSlangByDefault) }
```

Keeps the current result's expansion locally. Initialize from the saved default when the word/default changes; a local expand/collapse is not a preference write.

Line 29

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 30

```
Surface(modifier = Modifier.fillMaxSize(), color =
Color(0xFFF9F9FB)) {
```

A visual container providing background, border and shape.

Line 31

```
Column(modifier = Modifier.fillMaxSize().padding(24.dp))
{
```

A vertical layout: the children below are arranged top to bottom.

Line 32

```
if (word.isNotBlank()) {
```

Checks this condition and runs the following branch only when it is true: (word.isNotBlank()).

Line 33

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 34

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 35

```
horizontalArrangement = Arrangement.SpaceBetween,
```

Controls how children are spread horizontally.

Line 36

```
verticalAlignment = Alignment.Bottom
```

Controls vertical alignment of children.

Line 37

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 38

```
Text(
```

Draws text using the supplied value/style.

Line 39

```
text = word.uppercase(),
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 40

```
style = MaterialTheme.typography.headlineMedium.copy(
```

Chooses a theme text style or a customized copy of it.

Line 41

```
fontWeight = FontWeight.Black,
```

Sets how bold the text is.

Line 42

```
letterSpacing = 1.sp
```

Sets the spacing between letters.

Line 43

```
),
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 44

```
color = Color.Black
```

Sets this element's color.

Line 45

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
Text(
```

Draws text using the supplied value/style.

Line 47

```
text = "QUERY_RESOLVED",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 48

```
style = MaterialTheme.typography.labelSmall,
```

Chooses a theme text style or a customized copy of it.

Line 49

```
color = Color.Gray
```

Sets this element's color.

Line 50

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 51

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 52

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 53

```
HorizontalDivider(color = Color.Black, thickness = 2.dp)
```

Draws a dividing line.

Line 54

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 55

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 56

```
if (word.isNotBlank()) {
```

Checks this condition and runs the following branch only when it is true: (word.isNotBlank()).

Line 57

```
TextButton(onClick = onRefresh, enabled = state !is
UiState.Loading) {
```

Draws a spinner while waiting for a search.

Line 58

```
Text("Refresh definitions")
```

Draws text using the supplied value/style.

Line 59

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 60

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 61

```
Spacer(modifier = Modifier.height(16.dp))
```

Adds empty space between UI elements.

Line 62

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 63

```
Box(modifier = Modifier.fillMaxSize()) {
```

A container that can position/stack its children.

Line 64

```
when (state) {
```

Selects the UI for the complete result state family.

Line 65

```
is UiState.Loading -> CircularProgressIndicator(
```

Draws a spinner while waiting for a search.

Line 66

```
modifier = Modifier.align(Alignment.Center),
```

Applies the following layout/appearance instructions to this UI element.

Line 67

```
color = Color.Black
```

Sets this element’s color.

Line 68

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 69

```
is UiState.Error -> Text(
```

Displays the readable error carried by this state.

Line 70

```
text = "ERROR: ${state.message}",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 71

```
color = Color.Red,
```

Sets this element’s color.

Line 72

```
modifier = Modifier.align(Alignment.Center)
```

Applies the following layout/appearance instructions to this UI element.

Line 73

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 74

```
is UiState.Success -> {
```

Starts the list that displays available definitions and suggestions.

Line 75

```
LazyColumn(
```

Draws a scrollable vertical list that creates items as needed.

Line 76

```
modifier = Modifier.fillMaxSize(),
```

Applies the following layout/appearance instructions to this UI element.

Line 77

```
verticalArrangement = Arrangement.spacedBy(12.dp)
```

Controls how children are spread vertically.

Line 78

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 79

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 80

```
// Render "Did you mean...?" banner if Merriam-Webster
offered a correction
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 81

```
if (!state.didYouMeanSuggestion.isNullOrBlank()) {
```

Checks this condition and runs the following branch only when it is true: (! state.didYouMeanSuggestion.isNullOrBlank()).

Line 82

```
item {
```

Adds one non-repeating item to this LazyColumn, such as a heading, suggestion banner or slang section.

Line 83

```
Surface(
```

A visual container providing background, border and shape.

Line 84

```
modifier = Modifier
```

Applies the following layout/appearance instructions to this UI element.

Line 85

```
.fillMaxWidth()
```

Uses the maximum available width in the parent layout.

Line 86

```
.clickable {
onSearchSuggestion(state.didYouMeanSuggestion) },
```

Reports the suggested word to App so it can navigate to that result.

Line 87

```
color = Color(0xFFE3F2FD),
```

Sets this element’s color.

Line 88

```
border = BorderStroke(1.dp, Color(0xFF90CAF9)),
```

Sets the outline width and color.

Line 89

```
shape = Shapes().extraSmall
```

Chooses the component’s corner shape.

Line 90

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 91

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 92

```
modifier = Modifier.padding(14.dp),
```

Applies the following layout/appearance instructions to this UI element.

Line 93

```
verticalAlignment = Alignment.CenterVertically
```

Controls vertical alignment of children.

Line 94

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 95

```
Text("💡", fontSize = 16.sp)
```

Draws text using the supplied value/style.

Line 96

```
Spacer(modifier = Modifier.width(12.dp))
```

Adds empty space between UI elements.

Line 97

```
Column {
```

Starts a vertical layout using its default configuration; the next children are drawn from top to bottom.

Line 98

```
Text(
```

Draws text using the supplied value/style.

Line 99

```
"Did you mean:
${state.didYouMeanSuggestion.uppercase()}?",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 100

```
style =
MaterialTheme.typography.labelLarge.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 101

```
color = Color(0xFF0D47A1)
```

Sets this element's color.

Line 102

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 103

```
Text(
```

Draws text using the supplied value/style.

Line 104

```
"Click here to search the corrected spelling.",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 105

```
style = MaterialTheme.typography.bodySmall,
```

Chooses a theme text style or a customized copy of it.

Line 106

```
color = Color(0xFF1565C0)
```

Sets this element's color.

Line 107

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 108

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 109

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 110

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 111

```
Spacer(modifier = Modifier.height(4.dp))
```

Adds empty space between UI elements.

Line 112

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 113

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 114

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 115

```
if (state.formalEntries.isNotEmpty()) {
```

Checks this condition and runs the following branch only when it is true: (state.formalEntries.isNotEmpty()).

Line 116

```
item {
```

Adds one non-repeating item to this LazyColumn, such as a heading, suggestion banner or slang section.

Line 117

```
Text(
```

Draws text using the supplied value/style.

Line 118

```
"// FORMAL DEFINITIONS",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 119

```
style =
MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 120

```
color = Color.DarkGray
```

Sets this element’s color.

Line 121

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 123

```
items(state.formalEntries) { entry ->
```

Creates one formal definition card for each formal entry.

Line 124

```
SharpEntryCard(entry, isSlang = false)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 125

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 126

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 127

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 128

```
if (state.urbanEntries.isNotEmpty()) {
```

Checks this condition and runs the following branch only when it is true: (state.urbanEntries.isNotEmpty()).

Line 129

```
item {
```

Adds one non-repeating item to this LazyColumn, such as a heading, suggestion banner or slang section.

Line 130

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 131

```
Surface(
```

A visual container providing background, border and shape.

Line 132

```
modifier = Modifier
```

Applies the following layout/appearance instructions to this UI element.

Line 133

```
.fillMaxWidth()
```

Uses the maximum available width in the parent layout.

Line 134

```
.clickable { showUrbanSlang = !showUrbanSlang },
```

Expands/collapses slang locally on this result; it does not overwrite the saved default.

Line 135

```
color = Color(0xFFFFF3CD),
```

Sets this element's color.

Line 136

```
border = BorderStroke(1.dp, Color(0xFFFFEEBA)),
```

Sets the outline width and color.

Line 137

```
shape = Shapes().extraSmall
```

Chooses the component's corner shape.

Line 138

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 139

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 140

```
modifier = Modifier.padding(16.dp),
```

Applies the following layout/appearance instructions to this UI element.

Line 141

```
verticalAlignment = Alignment.CenterVertically
```

Controls vertical alignment of children.

Line 142

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 143

```
Text("⚠️", fontSize = 16.sp)
```

Draws text using the supplied value/style.

Line 144

```
Spacer(modifier = Modifier.width(12.dp))
```

Adds empty space between UI elements.

Line 145

```
Column(modifier = Modifier.weight(1f)) {
```

A vertical layout: the children below are arranged top to bottom.

Line 146

```
Text(
```

Draws text using the supplied value/style.

Line 147

```
"URBAN SLANG MATRIX",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 148

```
style =
MaterialTheme.typography.labelLarge.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 149

```
color = Color(0xFF856404)
```

Sets this element's color.

Line 150

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 151

```
Text(
```

Draws text using the supplied value/style.

Line 152

```
if (showUrbanSlang) "Click to collapse unstructured
dataset" else "Contains unfiltered explicit terms. Click
to expand.",
```

Checks this condition and runs the following branch only when it is true: (showUrbanSlang) "Click to collapse unstructured dataset" else "Contains unfiltered explicit terms. Click to expand.",.

Line 153

```
style = MaterialTheme.typography.bodySmall,
```

Chooses a theme text style or a customized copy of it.

Line 154

```
color = Color(0xFF856404)
```

Sets this element’s color.

Line 155

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 156

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 157

```
Text(if (showUrbanSlang) "[HIDE]" else "[SHOW]",
fontWeight = FontWeight.Bold, color = Color(0xFF856404))
```

Draws text using the supplied value/style.

Line 158

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 159

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 160

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 161

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 162

```
item {
```

Adds one non-repeating item to this LazyColumn, such as a heading, suggestion banner or slang section.

Line 163

```
Column(
```

A vertical layout: the children below are arranged top to bottom.

Line 164

```
modifier = Modifier.fillMaxWidth()
```

Applies the following layout/appearance instructions to this UI element.

Line 165

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 166

```
if (showUrbanSlang) {
```

Checks this condition and runs the following branch only when it is true: (showUrbanSlang).

Line 167

```
Spacer(modifier = Modifier.height(12.dp))
```

Adds empty space between UI elements.

Line 168

```
Text(
```

Draws text using the supplied value/style.

Line 169

```
"// SLANG ENTRIES",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 170

```
style =
MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 171

```
color = Color.DarkGray
```

Sets this element's color.

Line 172

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 173

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 174

```
Column(verticalArrangement = Arrangement.spacedBy(12.dp))
 {
```

A vertical layout: the children below are arranged top to bottom.

Line 175

```
state.urbanEntries.forEach { entry ->
```

Creates a card for each slang entry when that section is expanded.

Line 176

```
SharpEntryCard(entry, isSlang = true)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 177

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 178

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 179

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 180

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 181

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 182

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 183

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 184

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 185

```
is UiState.Idle -> Text(
```

Displays the initial guidance before a word result exists.

Line 186

```
"No active query. Input a target term.",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 187

```
modifier = Modifier.align(Alignment.Center),
```

Applies the following layout/appearance instructions to this UI element.

Line 188

```
color = Color.Gray
```

Sets this element’s color.

Line 189

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 190

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 191

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 192

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 193

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 194

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 195

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 196

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 197

```
fun SharpEntryCard(entry: DictionaryEntry, isSlang:
Boolean) {
```

Defines the reusable visual card for one DictionaryEntry.

Line 198

```
Surface(
```

A visual container providing background, border and shape.

Line 199

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 200

```
color = Color.White,
```

Sets this element’s color.

Line 201

```
border = BorderStroke(1.dp, Color.Black),
```

Sets the outline width and color.

Line 202

```
shape = Shapes().extraSmall
```

Chooses the component’s corner shape.

Line 203

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 204

```
Column(modifier = Modifier.padding(16.dp)) {
```

A vertical layout: the children below are arranged top to bottom.

Line 205

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 206

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 207

```
horizontalArrangement = Arrangement.SpaceBetween,
```

Controls how children are spread horizontally.

Line 208

```
verticalAlignment = Alignment.CenterVertically
```

Controls vertical alignment of children.

Line 209

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 210

```
Text(
```

Draws text using the supplied value/style.

Line 211

```
text = "[ ${entry.source.uppercase()} ]",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 212

```
style =
MaterialTheme.typography.labelSmall.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 213

```
color = if (isSlang) Color(0xFFD32F2F) else Color.Black
```

Sets this element’s color.

Line 214

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 215

```
if (entry.partOfSpeech.isNotEmpty()) {
```

Checks this condition and runs the following branch only when it is true: (entry.partOfSpeech.isNotEmpty()).

Line 216

```
Text(
```

Draws text using the supplied value/style.

Line 217

```
text = entry.partOfSpeech,
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 218

```
style = MaterialTheme.typography.labelSmall,
```

Chooses a theme text style or a customized copy of it.

Line 219

```
color = Color.Gray
```

Sets this element’s color.

Line 220

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 221

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 222

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 223

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 224

```
Text(
```

Draws text using the supplied value/style.

Line 225

```
text = entry.definition,
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 226

```
style =
MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Normal),
```

Chooses a theme text style or a customized copy of it.

Line 227

```
color = Color.Black
```

Sets this element's color.

Line 228

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 229

```
if (entry.example.isNotEmpty()) {
```

Checks this condition and runs the following branch only when it is true: (entry.example.isNotEmpty()).

Line 230

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 231

```
Box(
```

A container that can position/stack its children.

Line 232

```
modifier = Modifier
```

Applies the following layout/appearance instructions to this UI element.

Line 233

```
.fillMaxWidth()
```

Uses the maximum available width in the parent layout.

Line 234

```
.background(Color(0xFFF4F4F6))
```

Paints the specified background behind this component.

Line 235

```
.padding(10.dp)
```

Adds spacing inside/around this component using the specified dp values.

Line 236

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 237

```
Text(
```

Draws text using the supplied value/style.

Line 238

```
text = "EX: \"${entry.example}\"",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 239

```
style = MaterialTheme.typography.bodySmall,
```

Chooses a theme text style or a customized copy of it.

Line 240

```
color = Color.DarkGray
```

Sets this element’s color.

Line 241

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 242

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 243

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 244

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 245

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 246

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

10. HistoryScreen.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/ui/HistoryScreen.kt

Draws already-observed history and reports row/star taps.

Line 1

```
package com.example.dictionary.ui
```

Places this file in the com.example.dictionary.ui namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.foundation.BorderStroke
```

Makes BorderStroke available here from androidx.compose.foundation.

Line 4

```
import androidx.compose.foundation.clickable
```

Makes clickable available here from androidx.compose.foundation.

Line 5

```
import androidx.compose.foundation.layout.*
```

Makes * available from androidx.compose.foundation.layout. A star imports accessible declarations from that package.

Line 6

```
import androidx.compose.foundation.lazy.LazyColumn
```

Makes LazyColumn available here from androidx.compose.foundation.lazy.

Line 7

```
import androidx.compose.foundation.lazy.items
```

Makes items available here from androidx.compose.foundation.lazy.

Line 8

```
import androidx.compose.material3.*
```

Makes * available from androidx.compose.material3. A star imports accessible declarations from that package.

Line 9

```
import androidx.compose.runtime.*
```

Makes * available from androidx.compose.runtime. A star imports accessible declarations from that package.

Line 10

```
import androidx.compose.ui.Alignment
```

Makes Alignment available here from androidx.compose.ui.

Line 11

```
import androidx.compose.ui.Modifier
```

Makes Modifier available here from androidx.compose.ui.

Line 12

```
import androidx.compose.ui.graphics.Color
```

Makes Color available here from androidx.compose.ui.graphics.

Line 13

```
import androidx.compose.ui.text.font.FontWeight
```

Makes FontWeight available here from androidx.compose.ui.text.font.

Line 14

```
import androidx.compose.ui.unit.dp
```

Makes dp available here from androidx.compose.ui.unit.

Line 15

```
import com.example.dictionary.domain.HistoryItem
```

Makes HistoryItem available here from com.example.dictionary.domain.

Line 16

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 17

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 18

```
fun HistoryScreen(
```

Defines HistoryScreen(). Inputs are inside parentheses; the return type describes its answer.

Line 19

```
history: List<HistoryItem>,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 20

```
onSelectWord: (String) -> Unit,
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 21

```
onToggleFavorite: (String) -> Unit
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 22

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 23

```
Surface(modifier = Modifier.fillMaxSize(), color =
Color(0xFFF9F9FB)) {
```

A visual container providing background, border and shape.

Line 24

```
Column(modifier = Modifier.fillMaxSize().padding(24.dp))
{
```

A vertical layout: the children below are arranged top to bottom.

Line 25

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 26

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 27

```
horizontalArrangement = Arrangement.SpaceBetween,
```

Controls how children are spread horizontally.

Line 28

```
verticalAlignment = Alignment.Bottom
```

Controls vertical alignment of children.

Line 29

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 30

```
Text(
```

Draws text using the supplied value/style.

Line 31

```
text = "SEARCH_LEDGER",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 32

```
style =
MaterialTheme.typography.headlineSmall.copy(fontWeight =
FontWeight.Black),
```

Chooses a theme text style or a customized copy of it.

Line 33

```
color = Color.Black
```

Sets this element’s color.

Line 34

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 35

```
Text(
```

Draws text using the supplied value/style.

Line 36

```
text = "${history.size} ENTRIES",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 37

```
style = MaterialTheme.typography.labelSmall,
```

Chooses a theme text style or a customized copy of it.

Line 38

```
color = Color.Gray
```

Sets this element’s color.

Line 39

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 40

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 41

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 42

```
HorizontalDivider(color = Color.Black, thickness = 2.dp)
```

Draws a dividing line.

Line 43

```
Spacer(modifier = Modifier.height(16.dp))
```

Adds empty space between UI elements.

Line 44

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 45

```
if (history.isEmpty()) {
```

Shows the empty-history message when there are no saved rows.

Line 46

```
Box(modifier = Modifier.fillMaxSize(), contentAlignment =
 Alignment.Center) {
```

A container that can position/stack its children.

Line 47

```
Text("No saved searches yet.", style =
MaterialTheme.typography.bodyMedium, color = Color.Gray)
```

Draws text using the supplied value/style.

Line 48

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 49

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 50

```
LazyColumn(modifier = Modifier.fillMaxSize(),
verticalArrangement = Arrangement.spacedBy(8.dp)) {
```

Draws a scrollable vertical list that creates items as needed.

Line 51

```
items(history) { item ->
```

Draws one history row per saved item; the list has already been ordered by Room.

Line 52

```
Surface(
```

A visual container providing background, border and shape.

Line 53

```
modifier = Modifier
```

Applies the following layout/appearance instructions to this UI element.

Line 54

```
.fillMaxWidth()
```

Uses the maximum available width in the parent layout.

Line 55

```
.clickable { onSelectWord(item.word) },
```

Tells App which saved word was tapped, so it can open that result.

Line 56

```
color = if (item.isFavorite) Color(0xFFFFF9C4) else
Color.White,
```

Only displays the extra star badge for favorite items.

Line 57

```
border = BorderStroke(1.dp, Color.Black),
```

Sets the outline width and color.

Line 58

```
shape = Shapes().extraSmall
```

Chooses the component’s corner shape.

Line 59

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 60

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 61

```
modifier = Modifier.padding(16.dp).fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 62

```
horizontalArrangement = Arrangement.SpaceBetween,
```

Controls how children are spread horizontally.

Line 63

```
verticalAlignment = Alignment.CenterVertically
```

Controls vertical alignment of children.

Line 64

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 65

```
Column(modifier = Modifier.weight(1f)) {
```

A vertical layout: the children below are arranged top to bottom.

Line 66

```
Row(verticalAlignment = Alignment.CenterVertically) {
```

A horizontal layout: the children below are arranged side by side.

Line 67

```
Text(
```

Draws text using the supplied value/style.

Line 68

```
text = item.word.uppercase(),
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 69

```
style =
MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 70

```
color = Color.Black
```

Sets this element’s color.

Line 71

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 72

```
if (item.isFavorite) {
```

Only displays the extra star badge for favorite items.

Line 73

```
Spacer(modifier = Modifier.width(8.dp))
```

Adds empty space between UI elements.

Line 74

```
Text("★", color = Color(0xFFF57F17))
```

Draws text using the supplied value/style.

Line 75

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 76

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 77

```
Spacer(modifier = Modifier.height(4.dp))
```

Adds empty space between UI elements.

Line 78

```
Text(
```

Draws text using the supplied value/style.

Line 79

```
text = item.snippet,
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 80

```
style = MaterialTheme.typography.bodySmall,
```

Chooses a theme text style or a customized copy of it.

Line 81

```
color = Color.DarkGray,
```

Sets this element's color.

Line 82

```
maxLines = 1
```

Limits how many lines this text can occupy.

Line 83

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 84

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 85

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 86

```
IconButton(onClick = { onToggleFavorite(item.word) }) {
```

Tells App which word's star should be changed. The database update occurs outside this UI file.

Line 87

```
Text(if (item.isFavorite) "★" else "☆", style =
MaterialTheme.typography.titleLarge)
```

Only displays the extra star badge for favorite items.

Line 88

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 89

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 90

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 91

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 92

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 93

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 94

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 95

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 96

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

11. HistoryUseCase.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/HistoryUseCase.kt

A domain-facing way to observe history and change a favorite through the repository.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
class HistoryUseCase(private val repository:
DictionaryRepository) {
```

Defines HistoryUseCase as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 6

```
fun observeHistory(): Flow<List<HistoryItem>> =
repository.observeHistory()
```

Defines observeHistory(). Inputs are inside parentheses; the return type describes its answer.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
suspend fun toggleFavorite(word: String) {
```

Defines toggleFavorite(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 9

```
repository.toggleFavorite(word)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

12. PreferencesRepository.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/PreferencesRepository.kt

The contract for observing/saving the slang-default Boolean.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
interface PreferencesRepository {
```

Defines PreferencesRepository as an interface of promised operations.

Line 6

```
fun observeShowSlang(): Flow<Boolean>
```

Defines observeShowSlang(). Inputs are inside parentheses; the return type describes its answer.

Line 7

```
suspend fun saveShowSlang(enabled: Boolean)
```

Defines saveShowSlang(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

13. SlangPreferenceUseCase.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/SlangPreferenceUseCase.kt

Forwards preference operations through the preference interface.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
class SlangPreferenceUseCase(private val preferences:
PreferencesRepository) {
```

Defines SlangPreferenceUseCase as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 6

```
fun observeShowSlang(): Flow<Boolean> =
preferences.observeShowSlang()
```

Defines observeShowSlang(). Inputs are inside parentheses; the return type describes its answer.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
suspend fun saveShowSlang(enabled: Boolean) {
```

Defines saveShowSlang(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 9

```
preferences.saveShowSlang(enabled)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

14. HistoryEntity.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/local/HistoryEntity.kt

Defines one saved-search database row, including its primary-key word.

Line 1

```
package com.example.dictionary.data.local
```

Places this file in the com.example.dictionary.data.local namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.room3.Entity
```

Makes Entity available here from androidx.room3.

Line 4

```
import androidx.room3.PrimaryKey
```

Makes PrimaryKey available here from androidx.room3.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
// Module 9: one saved search per word. Room stores this
in SQLite.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 7

```
@Entity(tableName = "search_history")
```

Registers the next Kotlin data class as a Room database table.

Line 8

```
data class HistoryEntity(
```

Defines HistoryEntity as a data container.

Line 9

```
@PrimaryKey val word: String,
```

The word associated with this object.

Line 10

```
val snippet: String,
```

A short preview of a saved definition for History.

Line 11

```
val resultJson: String,
```

Serialized definition lists stored as one database text value.

Line 12

```
val isFavorite: Boolean = false,
```

Whether this history word is starred; false is the default.

Line 13

```
val searchOrder: Long = 0
```

A numeric order used to display newer saved/refreshed searches first.

Line 14

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

15. HistoryDao.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/local/HistoryDao.kt

Room’s saved-word operations. SQL chooses rows, updates definitions and flips stars.

Line 1

```
package com.example.dictionary.data.local
```

Places this file in the com.example.dictionary.data.local namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.room3.Dao
```

Makes Dao available here from androidx.room3.

Line 4

```
import androidx.room3.Query
```

Makes Query available here from androidx.room3.

Line 5

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 6

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 7

```
@Dao
```

Asks Room to generate the database implementation for the following DAO interface.

Line 8

```
interface HistoryDao {
```

Defines HistoryDao as an interface of promised operations.

Line 9

```
@Query("SELECT * FROM search_history ORDER BY isFavorite
DESC, searchOrder DESC")
```

Defines the history query: favorites first, then most recently saved/refreshed rows.

Line 10

```
fun observeHistory(): Flow<List<HistoryEntity>>
```

Room returns a Flow that emits again when the table changes.

Line 11

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 12

```
@Query("SELECT * FROM search_history WHERE word = :word")
```

Defines a lookup for exactly one saved word.

Line 13

```
suspend fun findWord(word: String): HistoryEntity?
```

Returns that row, or null when the word is not stored.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
// Updating a definition keeps the user's existing
favorite flag.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 16

```
@Query("""
```

Attaches this SQL statement to the DAO function immediately below it. Room checks it and generates the database call.

Line 17

```
INSERT INTO search_history (word, snippet, resultJson,
isFavorite, searchOrder)
```

Names the columns written for a saved search.

Line 18

```
VALUES (:word, :snippet, :resultJson, 0,
```

Supplies query parameters, sets a new row's favorite to false, and begins calculating its order.

Line 19

```
(SELECT COALESCE(MAX(searchOrder), 0) + 1 FROM
search_history))
```

Uses the highest existing order plus one; COALESCE uses zero for an empty table.

Line 20

```
ON CONFLICT(word) DO UPDATE SET
```

If this primary-key word exists, update that row instead of creating a duplicate.

Line 21

```
snippet = excluded.snippet,
```

Replace the existing row's preview with the new preview.

Line 22

```
resultJson = excluded.resultJson,
```

Replace saved definitions with the new serialized result.

Line 23

```
searchOrder = excluded.searchOrder
```

Move the saved/refreshed word to the newest order within its favorite group. isFavorite is deliberately not overwritten.

Line 24

```
""")
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 25

```
suspend fun saveWord(word: String, snippet: String,
resultJson: String)
```

Declares the parameters Room binds into the insertion/update query above.

Line 26

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 27

```
@Query("UPDATE search_history SET isFavorite = NOT
isFavorite WHERE word = :word")
```

Flips the favorite flag only for the matching word.

Line 28

```
suspend fun toggleFavorite(word: String)
```

Runs the favorite update asynchronously through Room.

Line 29

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

16. DictionaryDatabase.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/local/DictionaryDatabase.kt

Registers the entity and exposes its DAO. Room generates the constructor implementation.

Line 1

```
package com.example.dictionary.data.local
```

Places this file in the com.example.dictionary.data.local namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.room3.ConstructedBy
```

Makes ConstructedBy available here from androidx.room3.

Line 4

```
import androidx.room3.Database
```

Makes Database available here from androidx.room3.

Line 5

```
import androidx.room3.RoomDatabase
```

Makes RoomDatabase available here from androidx.room3.

Line 6

```
import androidx.room3.RoomDatabaseConstructor
```

Makes RoomDatabaseConstructor available here from androidx.room3.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
@Database(entities = [HistoryEntity::class], version = 1)
```

Registers HistoryEntity in database schema version 1.

Line 9

```
@ConstructedBy(DictionaryDatabaseConstructor::class)
```

Connects this database to the constructor Room generates for the platforms.

Line 10

```
abstract class DictionaryDatabase : RoomDatabase() {
```

Defines DictionaryDatabase as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 11

```
abstract fun historyDao(): HistoryDao
```

Defines historyDao(). Inputs are inside parentheses; the return type describes its answer.

Line 12

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 13

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 14

```
// Room generates the actual implementation for each
target.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 15

```
@Suppress("KotlinNoActualForExpect")
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 16

```
expect object DictionaryDatabaseConstructor :
RoomDatabaseConstructor<DictionaryDatabase> {
```

Declares the database constructor that Room will generate for each target. This is generated-library setup, not a manually implemented dictionary feature.

Line 17

```
override fun initialize(): DictionaryDatabase
```

Defines initialize(). This fulfills the matching interface/parent operation. Inputs are inside parentheses; the return type describes its answer.

Line 18

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

17. DictionaryPreferences.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/DictionaryPreferences.kt

Reads and saves one small setting using DataStore.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStore
```

Makes DataStore available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.preferences.core.Preferences
```

Makes Preferences available here from androidx.datastore.preferences.core.

Line 5

```
import
androidx.datastore.preferences.core.booleanPreferencesKey
```

Makes booleanPreferencesKey available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.datastore.preferences.core.edit
```

Makes edit available here from androidx.datastore.preferences.core.

Line 7

```
import
com.example.dictionary.domain.PreferencesRepository
```

Makes PreferencesRepository available here from com.example.dictionary.domain.

Line 8

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 9

```
import kotlinx.coroutines.flow.map
```

Makes map available here from kotlinx.coroutines.flow.

Line 10

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 11

```
// Module 9: DataStore is for a small setting;
definitions belong in Room.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 12

```
class DictionaryPreferences(private val dataStore:
DataStore<Preferences>) : PreferencesRepository {
```

Implements the preference interface using the injected DataStore object.

Line 13

```
private val showSlangKey =
booleanPreferencesKey("show_slang")
```

Defines the Boolean key named show_slang used in the preference file.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
override fun observeShowSlang(): Flow<Boolean> {
```

Returns a stream of the saved Boolean preference.

Line 16

```
return dataStore.data.map { preferences ->
preferences[showSlangKey] ?: false }
```

Reads the key from each preference snapshot, using false when the user has never saved it.

Line 17

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 19

```
override suspend fun saveShowSlang(enabled: Boolean) {
```

Declares the operation to save a new Boolean preference.

Line 20

```
dataStore.edit { preferences -> preferences[showSlangKey]
 = enabled }
```

Changes only this preference key in a DataStore update.

Line 21

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 22

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

18. AppStorage.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/AppStorage.kt

A small container holding the database and preferences supplied by a platform entry point.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStore
```

Makes DataStore available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.preferences.core.Preferences
```

Makes Preferences available here from androidx.datastore.preferences.core.

Line 5

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 6

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 7

```
// Platform entry points create these once and inject
them into App.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 8

```
class AppStorage(
```

Defines AppStorage as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 9

```
val database: DictionaryDatabase,
```

Declares the database value and its type for this object or operation. A question mark allows null; a default is used if no value is supplied.

Line 10

```
val preferences: DataStore<Preferences>
```

Receives/keeps preferences as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 11

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

19. HttpClientFactory.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/HttpClientFactory.kt

Declares the platform factory and centralizes JSON, logs and timeouts.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.HttpClientConfig
```

Makes HttpClientConfig available here from io.ktor.client.

Line 5

```
import io.ktor.client.plugins.HttpTimeout
```

Makes HttpTimeout available here from io.ktor.client.plugins.

Line 6

```
import io.ktor.client.plugins.contentnegotiation.ContentN
egotiation
```

Makes ContentNegotiation available here from io.ktor.client.plugins.contentnegotiation.

Line 7

```
import io.ktor.client.plugins.logging.Logging
```

Makes Logging available here from io.ktor.client.plugins.logging.

Line 8

```
import io.ktor.client.plugins.logging.LogLevel
```

Makes LogLevel available here from io.ktor.client.plugins.logging.

Line 9

```
import io.ktor.serialization.kotlinx.json.json
```

Makes json available here from io.ktor.serialization.kotlinx.json.

Line 10

```
import kotlinx.serialization.json.Json
```

Makes Json available here from kotlinx.serialization.json.

Line 11

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 12

```
// Module 6: each platform chooses its own networking
engine.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 13

```
expect fun createHttpClient(): HttpClient
```

Shared declaration: each platform must supply an actual HTTP client factory.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
// Module 9: the plugins are configured once in shared
code.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 16

```
fun HttpClientConfig<*>.configureDictionaryClient() {
```

An extension function: adds shared configuration steps to any Ktor client engine configuration.

Line 17

```
install(ContentNegotiation) {
```

Adds the plugin that converts HTTP response bodies using the configured serializer.

Line 18

```
json(Json { ignoreUnknownKeys = true })
```

Uses Kotlin JSON serialization and ignores extra response fields.

Line 19

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 20

```
install(Logging) {
```

Adds Ktor’s request/response logging support.

Line 21

```
level = LogLevel.INFO
```

Logs general request/response information rather than response bodies.

Line 22

```
// The other API's URL contains a key, so log only
key-free requests.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 23

```
filter { request -> request.url.host ==
"api.urbandictionary.com" }
```

Logs only Urban Dictionary traffic; formal request URLs contain the API key.

Line 24

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 25

```
install(HttpTimeout) {
```

Adds time limits so requests do not wait indefinitely.

Line 26

```
requestTimeoutMillis = 10000
```

Limits the whole request to 10,000 milliseconds (10 seconds).

Line 27

```
connectTimeoutMillis = 10000
```

Limits establishing the connection to 10 seconds.

Line 28

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 29

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

20. build.gradle.kts

androidApp/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.kotlin.gradle.dsl.JvmTarget
```

Makes JvmTarget available here from org.jetbrains.kotlin.gradle.dsl.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
alias(libs.plugins.androidApplication)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
alias(libs.plugins.composeCompiler)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
kotlin {
```

Gradle setup: configures kotlin for the surrounding module/target. This runs during building, not when a user searches.

Line 9

```
compilerOptions {
```

Gradle setup: configures compilerOptions for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
jvmTarget = JvmTarget.JVM_11
```

Gradle setup: configures jvmTarget = JvmTarget.JVM_11 for the surrounding module/target. This runs during building, not when a user searches.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 12

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 13

```
dependencies {
```

Gradle setup: configures dependencies for the surrounding module/target. This runs during building, not when a user searches.

Line 14

```
implementation(project(":shared"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 15

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 16

```
implementation(libs.androidx.activity.compose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 17

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 18

```
implementation(libs.compose.uiToolingPreview)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 19

```
debugImplementation(libs.compose.uiTooling)
```

Gradle setup: configures debugImplementation(libs.compose.uiTooling) for the surrounding module/target. This runs during building, not when a user searches.

Line 20

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
android {
```

Gradle setup: configures android for the surrounding module/target. This runs during building, not when a user searches.

Line 23

```
namespace = "com.example.dictionary"
```

Gradle setup: configures namespace = "com.example.dictionary" for the surrounding module/target. This runs during building, not when a user searches.

Line 24

```
compileSdk =
libs.versions.android.compileSdk.get().toInt()
```

Gradle setup: configures compileSdk = libs.versions.android.compileSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 25

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 26

```
defaultConfig {
```

Gradle setup: configures defaultConfig for the surrounding module/target. This runs during building, not when a user searches.

Line 27

```
applicationId = "com.example.dictionary"
```

Gradle setup: configures applicationId = "com.example.dictionary" for the surrounding module/target. This runs during building, not when a user searches.

Line 28

```
minSdk = libs.versions.android.minSdk.get().toInt()
```

Gradle setup: configures minSdk = libs.versions.android.minSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 29

```
targetSdk = libs.versions.android.targetSdk.get().toInt()
```

Gradle setup: configures targetSdk = libs.versions.android.targetSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 30

```
versionCode = 1
```

Gradle setup: configures versionCode = 1 for the surrounding module/target. This runs during building, not when a user searches.

Line 31

```
versionName = "1.0"
```

Gradle setup: configures versionName = "1.0" for the surrounding module/target. This runs during building, not when a user searches.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 33

```
packaging {
```

Gradle setup: configures packaging for the surrounding module/target. This runs during building, not when a user searches.

Line 34

```
resources {
```

Gradle setup: configures resources for the surrounding module/target. This runs during building, not when a user searches.

Line 35

```
excludes += "/META-INF/{AL2.0,LGPL2.1}"
```

Gradle setup: configures excludes += "/META-INF/{AL2.0,LGPL2.1}" for the surrounding module/target. This runs during building, not when a user searches.

Line 36

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 37

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 38

```
buildTypes {
```

Gradle setup: configures buildTypes for the surrounding module/target. This runs during building, not when a user searches.

Line 39

```
release {
```

Gradle setup: configures release for the surrounding module/target. This runs during building, not when a user searches.

Line 40

```
isMinifyEnabled = false
```

Gradle setup: configures isMinifyEnabled = false for the surrounding module/target. This runs during building, not when a user searches.

Line 41

```
proguardFiles(
```

Gradle setup: configures proguardFiles(for the surrounding module/target. This runs during building, not when a user searches.

Line 42

```
getDefaultProguardFile("proguard-android-optimize.txt"),
```

Gradle setup: configures getDefaultProguardFile("proguard-android-optimize.txt"), for the surrounding module/target. This runs during building, not when a user searches.

Line 43

```
"proguard-rules.pro"
```

Gradle setup: configures "proguard-rules.pro" for the surrounding module/target. This runs during building, not when a user searches.

Line 44

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 45

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 47

```
compileOptions {
```

Gradle setup: configures compileOptions for the surrounding module/target. This runs during building, not when a user searches.

Line 48

```
sourceCompatibility = JavaVersion.VERSION_11
```

Gradle setup: configures sourceCompatibility = JavaVersion.VERSION_11 for the surrounding module/target. This runs during building, not when a user searches.

Line 49

```
targetCompatibility = JavaVersion.VERSION_11
```

Gradle setup: configures targetCompatibility = JavaVersion.VERSION_11 for the surrounding module/target. This runs during building, not when a user searches.

Line 50

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 51

```
buildFeatures {
```

Gradle setup: configures buildFeatures for the surrounding module/target. This runs during building, not when a user searches.

Line 52

```
compose = true
```

Gradle setup: configures compose = true for the surrounding module/target. This runs during building, not when a user searches.

Line 53

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 54

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

21. AndroidManifest.xml

androidApp/src/main/AndroidManifest.xml

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
<?xml version="1.0" encoding="utf-8"?>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
<manifest xmlns:android="http://schemas.android.com/apk/r
es/android">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 3

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 4

```
<uses-permission
android:name="android.permission.INTERNET" />
```

Allows the Android app to make Internet requests.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
<application
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 7

```
android:allowBackup="true"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 8

```
android:icon="@mipmap/ic_launcher"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 9

```
android:label="@string/app_name"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 10

```
android:roundIcon="@mipmap/ic_launcher_round"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 11

```
android:supportsRtl="true"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 12

```
android:theme="@android:style/Theme.Material.Light.NoActi
onBar">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 13

```
<activity
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 14

```
android:exported="true"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 15

```
android:name=".MainActivity">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 16

```
<intent-filter>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 17

```
<action android:name="android.intent.action.MAIN"/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 18

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 19

```
<category
android:name="android.intent.category.LAUNCHER"/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 20

```
</intent-filter>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 21

```
<intent-filter>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 22

```
<action android:name="android.intent.action.VIEW" />
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 23

```
<category android:name="android.intent.category.DEFAULT"
/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 24

```
<category
android:name="android.intent.category.BROWSABLE" />
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 25

```
<data android:scheme="dictionary" android:host="word" />
```

Registers dictionary://word/... as an Android app URL pattern.

Line 26

```
</intent-filter>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 27

```
</activity>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 28

```
</application>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 29

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 30

```
</manifest>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

22. MainActivity.kt

androidApp/src/main/kotlin/com/example/dictionary/MainActivity.kt

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import android.os.Bundle
```

Makes Bundle available here from android.os.

Line 4

```
import com.example.dictionary.data.createAppStorage
```

Makes createAppStorage available here from com.example.dictionary.data.

Line 5

```
import androidx.activity.ComponentActivity
```

Makes ComponentActivity available here from androidx.activity.

Line 6

```
import androidx.activity.compose.setContent
```

Makes setContent available here from androidx.activity.compose.

Line 7

```
import androidx.activity.enableEdgeToEdge
```

Makes enableEdgeToEdge available here from androidx.activity.

Line 8

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 9

```
class MainActivity : ComponentActivity() {
```

Defines MainActivity as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 10

```
override fun onCreate(savedInstanceState: Bundle?) {
```

Defines onCreate(). This fulfills the matching interface/parent operation. Inputs are inside parentheses; the return type describes its answer.

Line 11

```
enableEdgeToEdge()
```

Allows the Android content to draw edge-to-edge; Scaffold padding helps place content around its bars.

Line 12

```
super.onCreate(savedInstanceState)
```

Runs the Activity's parent initialization before showing the app.

Line 13

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 14

```
val storage = createAppStorage(applicationContext)
```

Creates/retains the platform's persistent storage objects before passing them into the shared App.

Line 15

```
setContent {
```

Makes the following Compose UI the Android Activity's content.

Line 16

```
App(storage)
```

Displays the shared Compose App and supplies the storage built by this platform.

Line 17

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 19

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

23. strings.xml

androidApp/src/main/res/values/strings.xml

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
<resources>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
<string name="app_name">dictionary</string>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 3

```
</resources>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

24. build.gradle.kts

browserStorage/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.kotlin.gradle.ExperimentalWasmDsl
```

Makes ExperimentalWasmDsl available here from org.jetbrains.kotlin.gradle.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
alias(libs.plugins.kotlinMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 6

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 7

```
kotlin {
```

Gradle setup: configures kotlin for the surrounding module/target. This runs during building, not when a user searches.

Line 8

```
js {
```

Gradle setup: configures js for the surrounding module/target. This runs during building, not when a user searches.

Line 9

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 12

```
@OptIn(ExperimentalWasmDsl::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 13

```
wasmJs {
```

Gradle setup: configures wasmJs for the surrounding module/target. This runs during building, not when a user searches.

Line 14

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 15

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 16

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 17

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 18

```
sourceSets {
```

Gradle setup: configures sourceSets for the surrounding module/target. This runs during building, not when a user searches.

Line 19

```
commonMain.dependencies {
```

Starts dependencies for commonMain. This determines which targets can see the following libraries.

Line 20

```
api(libs.androidx.sqlite.web)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 21

```
implementation(npm("dictionary-sqlite-worker",
layout.projectDirectory.dir("worker").asFile))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 22

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 23

```
wasmJsMain.dependencies {
```

Starts dependencies for wasmJsMain. This determines which targets can see the following libraries.

Line 24

```
implementation("org.jetbrains.kotlinx:kotlinx-browser:0.5
.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 25

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 26

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 27

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

25. BrowserDriver.kt (commonMain)

browserStorage/src/commonMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
package com.example.dictionary.browser
```

Places this file in the com.example.dictionary.browser namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.sqlite.driver.web.WebWorkerSQLiteDriver
```

Makes WebWorkerSQLiteDriver available here from androidx.sqlite.driver.web.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
// Browser-only infrastructure. App/domain/presentation
code does not depend on JS APIs.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 6

```
expect fun createBrowserDriver(): WebWorkerSQLiteDriver
```

Declares a shared operation that target-specific actual code must implement.

26. BrowserDriver.kt (jsMain)

browserStorage/src/jsMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
package com.example.dictionary.browser
```

Places this file in the com.example.dictionary.browser namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.sqlite.driver.web.WebWorkerSQLiteDriver
```

Makes WebWorkerSQLiteDriver available here from androidx.sqlite.driver.web.

Line 4

```
import org.w3c.dom.Worker
```

Makes Worker available here from org.w3c.dom.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createBrowserDriver(): WebWorkerSQLiteDriver {
```

Supplies this platform’s implementation for the shared expect declaration.

Line 7

```
return WebWorkerSQLiteDriver(Worker(js("""new
URL("dictionary-sqlite-worker/worker.js",
import.meta.url)""")))
```

Connects Room’s browser SQLite driver to a worker that runs database requests outside the UI thread.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

27. BrowserDriver.kt (wasmJsMain)

browserStorage/src/wasmJsMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
package com.example.dictionary.browser
```

Places this file in the com.example.dictionary.browser namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.sqlite.driver.web.WebWorkerSQLiteDriver
```

Makes WebWorkerSQLiteDriver available here from androidx.sqlite.driver.web.

Line 4

```
import org.w3c.dom.Worker
```

Makes Worker available here from org.w3c.dom.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createBrowserDriver(): WebWorkerSQLiteDriver =
 WebWorkerSQLiteDriver(createWorker())
```

Supplies this platform’s implementation for the shared expect declaration.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
@OptIn(ExperimentalWasmJsInterop::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 9

```
private fun createWorker(): Worker =
```

Defines createWorker(). Inputs are inside parentheses; the return type describes its answer.

Line 10

```
js("""new Worker(new
URL("dictionary-sqlite-worker/worker.js",
import.meta.url))""")
```

Browser-only interop: locates the bundled SQLite worker script and creates a worker to handle database requests.

28. package.json

browserStorage/worker/package.json

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
{
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 2

```
"name": "dictionary-sqlite-worker",
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 3

```
"version": "1.0.0",
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 4

```
"private": true,
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 5

```
"license": "Apache-2.0",
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 6

```
"dependencies": { "@sqlite.org/sqlite-wasm":
"3.50.1-build1" }
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

29. worker.js

browserStorage/worker/worker.js

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
// Adapted from danysantiago/room-web-demo (Apache-2.0);
see ../LICENSE.txt.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 2

```
import sqlite3InitModule from '@sqlite.org/sqlite-wasm';
```

Imports the SQLite WebAssembly initialization function from the worker package dependency.

Line 3

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 4

```
let sqlite3 = null;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
// Maps to track of active database connections and
prepared statements by their unique IDs.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 7

```
const databases = new Map(); // stores databaseId ->
SQLiteDbObject
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 8

```
const statements = new Map(); // stores statementId ->
SQLiteStatementObject
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
// Counters to generate unique IDs for new database
connections and statements.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 11

```
let nextDatabaseId = 0;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 12

```
let nextStatementId = 0;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 13

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 14

```
function openRequest(id, requestData) {
```

Defines a browser-worker helper that processes the request indicated by its name.

Line 15

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 16

```
const newDatabaseId = nextDatabaseId++;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 17

```
const newDatabase = new
sqlite3.oo1.OpfsDb(requestData.fileName);
```

Opens an SQLite database file inside the browser's Origin Private File System.

Line 18

```
databases.set(newDatabaseId, newDatabase);
```

Remembers the open database/statement under its generated numeric ID.

Line 19

```
postMessage({'id': id, data: {'databaseId':
newDatabaseId}});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 20

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 21

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 22

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 23

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 24

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 25

```
function prepareRequest(id, requestData) {
```

Defines a browser-worker helper that processes the request indicated by its name.

Line 26

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 27

```
const newStatementId = nextStatementId++;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 28

```
const resultData = {
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 29

```
'statementId': newStatementId,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 30

```
'parameterCount': 0,
```

Records how many parameter placeholders the prepared SQL expects.

Line 31

```
'columnNames': []
```

Builds the column-name information for the prepared statement reply.

Line 32

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 33

```
const database = databases.get(requestData.databaseId);
```

Finds the database/statement object matching the ID in this request.

Line 34

```
if (!database) {
```

Checks this condition and runs the following branch only when it is true: (! database).

Line 35

```
postMessage({'id': id, error: "Invalid database ID: " +
requestData.databaseId});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 36

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 37

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 38

```
const statement = database.prepare(requestData.sql);
```

Turns the supplied SQL into a prepared statement that can accept parameter values.

Line 39

```
statements.set(newStatementId, statement);
```

Remembers the open database/statement under its generated numeric ID.

Line 40

```
resultData.parameterCount =
sqlite3.capi.sqlite3_bind_parameter_count(statement);
```

Records how many parameter placeholders the prepared SQL expects.

Line 41

```
for (let i = 0; i < statement.columnCount; i++) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 42

```
resultData.columnNames.push(sqlite3.capi.sqlite3_column_n
ame(statement, i));
```

Builds the column-name information for the prepared statement reply.

Line 43

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 44

```
postMessage({'id': id, data: resultData});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 45

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 46

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 47

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 48

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 49

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 50

```
function stepRequest(id, requestData) {
```

Defines a browser-worker helper that processes the request indicated by its name.

Line 51

```
const statement =
statements.get(requestData.statementId);
```

Finds the database/statement object matching the ID in this request.

Line 52

```
if (!statement) {
```

Checks this condition and runs the following branch only when it is true: (! statement).

Line 53

```
postMessage({'id': id, error: "Invalid statement ID: " +
requestData.statementId});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 54

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 55

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 56

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 57

```
const resultData = {
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 58

```
'rows': [],
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 59

```
'columnTypes': []
```

Builds the result’s column-type information so the Kotlin driver can interpret returned values.

Line 60

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 61

```
statement.reset()
```

Resets the prepared statement before executing the next requested set of bindings.

Line 62

```
statement.clearBindings()
```

Clears previous parameter values before binding this request’s values.

Line 63

```
for (let i = 0; i < requestData.bindings.length; i++) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 64

```
statement.bind(i + 1, requestData.bindings[i]);
```

Binds this supplied value to the prepared SQL parameter, using SQLite’s one-based parameter positions.

Line 65

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 66

```
while (statement.step()) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 67

```
if (!resultData.columnTypes.length) {
```

Checks this condition and runs the following branch only when it is true: (! resultData.columnTypes.length).

Line 68

```
for (let i = 0; i < statement.columnCount; i++) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 69

```
resultData.columnTypes.push(sqlite3.capi.sqlite3_column_t
ype(statement, i));
```

Builds the result's column-type information so the Kotlin driver can interpret returned values.

Line 70

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 71

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 72

```
resultData.rows.push(statement.get([]));
```

Copies the current SQL result row into the reply's rows list.

Line 73

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 74

```
postMessage({'id': id, data: resultData});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 75

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 76

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 77

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 78

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 79

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 80

```
function closeRequest(id, requestData) {
```

Defines a browser-worker helper that processes the request indicated by its name.

Line 81

```
if (requestData.statementId != null) {
```

Checks this condition and runs the following branch only when it is true: (requestData.statementId != null).

Line 82

```
const statement =
statements.get(requestData.statementId);
```

Finds the database/statement object matching the ID in this request.

Line 83

```
if (!statement) {
```

Checks this condition and runs the following branch only when it is true: (! statement).

Line 84

```
postMessage({'id': id, error: "Invalid statement ID: " +
requestData.statementId});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 85

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 86

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 87

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 88

```
statement.finalize();
```

Releases the prepared SQL statement’s resources.

Line 89

```
statements.delete(requestData.statementId);
```

Removes the closed resource from the worker’s ID-to-object map.

Line 90

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 91

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 92

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 93

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 94

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 95

```
if (requestData.databaseId != null) {
```

Checks this condition and runs the following branch only when it is true: (requestData.databaseId != null).

Line 96

```
const database = databases.get(requestData.databaseId);
```

Finds the database/statement object matching the ID in this request.

Line 97

```
if (!database) {
```

Checks this condition and runs the following branch only when it is true: (! database).

Line 98

```
postMessage({'id': id, error: "Invalid database ID: " +
requestData.databaseId});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 99

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 100

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 101

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 102

```
database.close();
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 103

```
databases.delete(requestData.databaseId);
```

Removes the closed resource from the worker's ID-to-object map.

Line 104

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 105

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 106

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 107

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 108

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 109

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 110

```
// A map that links command names (strings) to their
respective handler functions.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 111

```
const commandMap = {
```

Maps a request command string to the function that processes it.

Line 112

```
'open': openRequest,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 113

```
'prepare': prepareRequest,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 114

```
'step': stepRequest,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 115

```
'close': closeRequest,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 116

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 117

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 118

```
function handleMessage(e) {
```

Dispatches this request to the matching open/prepare/step/close handler.

Line 119

```
const requestMsg = e.data;
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 120

```
if (!Object.hasOwn(requestMsg, 'data') || requestMsg.data
 == null) {
```

Checks this condition and runs the following branch only when it is true: (! Object.hasOwn(requestMsg, 'data') || requestMsg.data == null).

Line 121

```
postMessage(
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 122

```
{'id': requestMsg.id, 'error': "Invalid request, missing
'data'."}
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 123

```
);
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 124

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 125

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 126

```
if (!Object.hasOwn(requestMsg.data, 'cmd') ||
requestMsg.data.cmd == null) {
```

Checks this condition and runs the following branch only when it is true: (! Object.hasOwn(requestMsg.data, 'cmd') || requestMsg.data.cmd == null).

Line 127

```
postMessage(
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 128

```
{'id': requestMsg.id, 'error': "Invalid request, missing
'cmd'."}
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 129

```
);
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 130

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 131

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 132

```
const command = requestMsg.data.cmd;
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 133

```
const requestHandler = commandMap[command];
```

Maps a request command string to the function that processes it.

Line 134

```
if (requestHandler) {
```

Checks this condition and runs the following branch only when it is true: (requestHandler).

Line 135

```
requestHandler(requestMsg.id, requestMsg.data);
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 136

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 137

```
postMessage(
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 138

```
{'id': requestMsg.id, 'error': "Invalid request, unknown
command: '" + command + "'."}
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 139

```
);
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 140

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 141

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 142

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 143

```
const messageQueue = [];
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 144

```
onmessage = (e) => {
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 145

```
if (!sqlite3) {
```

Checks this condition and runs the following branch only when it is true: (! sqlite3).

Line 146

```
messageQueue.push(e);
```

Queues a request because SQLite has not finished initializing yet.

Line 147

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 148

```
handleMessage(e);
```

Dispatches this request to the matching open/prepare/step/close handler.

Line 149

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 150

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 151

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 152

```
sqlite3InitModule().then(instance => {
```

Initializes SQLite’s WebAssembly module. Queued database requests are handled after initialization succeeds.

Line 153

```
sqlite3 = instance;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 154

```
while (messageQueue.length > 0) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 155

```
handleMessage(messageQueue.shift());
```

Dispatches this request to the matching open/prepare/step/close handler.

Line 156

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 157

```
});
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

30. build.gradle.kts

build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 2

```
// this is necessary to avoid the plugins to be loaded
multiple times
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 3

```
// in each subproject's classloader
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 4

```
alias(libs.plugins.androidApplication) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
alias(libs.plugins.androidMultiplatformLibrary) apply
false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
alias(libs.plugins.composeMultiplatform) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 7

```
alias(libs.plugins.composeCompiler) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 8

```
alias(libs.plugins.kotlinJvm) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 9

```
alias(libs.plugins.kotlinMultiplatform) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

31. build.gradle.kts

desktopApp/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.compose.desktop.application.dsl.Targ
etFormat
```

Makes TargetFormat available here from org.jetbrains.compose.desktop.application.dsl.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
alias(libs.plugins.kotlinJvm)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
alias(libs.plugins.composeMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
alias(libs.plugins.composeCompiler)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 8

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 9

```
dependencies {
```

Gradle setup: configures dependencies for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
implementation(project(":shared"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 11

```
testImplementation(libs.kotlin.testJunit)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 12

```
testImplementation(libs.androidx.navigation.compose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 13

```
testImplementation(libs.androidx.lifecycle.viewmodelCompo
se)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 14

```
testImplementation(libs.androidx.lifecycle.runtimeCompose
)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 15

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 16

```
implementation(compose.desktop.currentOs)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 17

```
implementation(libs.kotlinx.coroutinesSwing)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 18

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 19

```
implementation(libs.compose.uiToolingPreview)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 20

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
compose.desktop {
```

Gradle setup: configures compose.desktop for the surrounding module/target. This runs during building, not when a user searches.

Line 23

```
application {
```

Gradle setup: configures application for the surrounding module/target. This runs during building, not when a user searches.

Line 24

```
mainClass = "com.example.dictionary.MainKt"
```

Gradle setup: configures mainClass = "com.example.dictionary.MainKt" for the surrounding module/target. This runs during building, not when a user searches.

Line 25

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 26

```
nativeDistributions {
```

Gradle setup: configures nativeDistributions for the surrounding module/target. This runs during building, not when a user searches.

Line 27

```
targetFormats(TargetFormat.Dmg, TargetFormat.Msi,
TargetFormat.Deb)
```

Gradle setup: configures targetFormats(TargetFormat.Dmg, TargetFormat.Msi, TargetFormat.Deb) for the surrounding module/target. This runs during building, not when a user searches.

Line 28

```
packageName = "com.example.dictionary"
```

Gradle setup: configures packageName = "com.example.dictionary" for the surrounding module/target. This runs during building, not when a user searches.

Line 29

```
packageVersion = "1.0.0"
```

Gradle setup: configures packageVersion = "1.0.0" for the surrounding module/target. This runs during building, not when a user searches.

Line 30

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 31

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

32. main.kt

desktopApp/src/main/kotlin/com/example/dictionary/main.kt

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.data.createAppStorage
```

Makes createAppStorage available here from com.example.dictionary.data.

Line 4

```
import androidx.compose.runtime.remember
```

Makes remember available here from androidx.compose.runtime.

Line 5

```
import androidx.compose.ui.window.Window
```

Makes Window available here from androidx.compose.ui.window.

Line 6

```
import androidx.compose.ui.window.application
```

Makes application available here from androidx.compose.ui.window.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
fun main() = application {
```

Defines main(). Inputs are inside parentheses; the return type describes its answer.

Line 9

```
val storage = remember { createAppStorage() }
```

Creates/retains the platform's persistent storage objects before passing them into the shared App.

Line 10

```
Window(
```

Creates the desktop application window; following lines configure its title and content.

Line 11

```
onCloseRequest = ::exitApplication,
```

Exits the desktop application when its window is closed.

Line 12

```
title = "dictionary",
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 13

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 14

```
App(storage)
```

Displays the shared Compose App and supplies the storage built by this platform.

Line 15

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 16

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

33. NavigationCompatibilityTest.kt

desktopApp/src/test/kotlin/com/example/dictionary/NavigationCompatibilityTest.kt

Automated checks with assertions. Study after the app logic; this code runs during tests, not normal searches.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.runtime.CompositionLocalProvider
```

Makes CompositionLocalProvider available here from androidx.compose.runtime.

Line 4

```
import androidx.compose.ui.ExperimentalComposeUiApi
```

Makes ExperimentalComposeUiApi available here from androidx.compose.ui.

Line 5

```
import androidx.compose.ui.ImageComposeScene
```

Makes ImageComposeScene available here from androidx.compose.ui.

Line 6

```
import androidx.lifecycle.Lifecycle
```

Makes Lifecycle available here from androidx.lifecycle.

Line 7

```
import androidx.lifecycle.LifecycleOwner
```

Makes LifecycleOwner available here from androidx.lifecycle.

Line 8

```
import androidx.lifecycle.LifecycleRegistry
```

Makes LifecycleRegistry available here from androidx.lifecycle.

Line 9

```
import androidx.lifecycle.ViewModelStore
```

Makes ViewModelStore available here from androidx.lifecycle.

Line 10

```
import androidx.lifecycle.ViewModelStoreOwner
```

Makes ViewModelStoreOwner available here from androidx.lifecycle.

Line 11

```
import androidx.lifecycle.compose.LocalLifecycleOwner
```

Makes LocalLifecycleOwner available here from androidx.lifecycle.compose.

Line 12

```
import androidx.lifecycle.viewmodel.compose.LocalViewMode
lStoreOwner
```

Makes LocalViewModelStoreOwner available here from androidx.lifecycle.viewmodel.compose.

Line 13

```
import androidx.navigation.NavHostController
```

Makes NavHostController available here from androidx.navigation.

Line 14

```
import androidx.navigation.compose.NavHost
```

Makes NavHost available here from androidx.navigation.compose.

Line 15

```
import androidx.navigation.compose.composable
```

Makes composable available here from androidx.navigation.compose.

Line 16

```
import androidx.navigation.compose.rememberNavController
```

Makes rememberNavController available here from androidx.navigation.compose.

Line 17

```
import androidx.savedstate.read
```

Makes read available here from androidx.savedstate.

Line 18

```
import javax.swing.SwingUtilities
```

Makes SwingUtilities available here from javax.swing.

Line 19

```
import kotlin.test.Test
```

Makes Test available here from kotlin.test.

Line 20

```
import kotlin.test.assertEquals
```

Makes assertEquals available here from kotlin.test.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
class NavigationCompatibilityTest {
```

Defines NavigationCompatibilityTest as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 23

```
@OptIn(ExperimentalComposeUiApi::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 24

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 25

```
fun navigationCanRestoreEntriesAndReadWordArguments() {
```

Defines navigationCanRestoreEntriesAndReadWordArguments(). Inputs are inside parentheses; the return type describes its answer.

Line 26

```
SwingUtilities.invokeAndWait {
```

Runs the desktop regression-test body on Swing's UI thread and waits for completion.

Line 27

```
val owner = object : LifecycleOwner, ViewModelStoreOwner
{
```

Stores a value/reference named owner from the expression on the right.

Line 28

```
override val lifecycle =
LifecycleRegistry.createUnsafe(this)
```

Stores a value/reference named lifecycle from the expression on the right.

Line 29

```
override val viewModelStore = ViewModelStore()
```

Stores a value/reference named viewModelStore from the expression on the right.

Line 30

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 31

```
owner.lifecycle.currentState = Lifecycle.State.RESUMED
```

Sets this test owner's lifecycle state so navigation can create and dispose entries correctly.

Line 32

```
var controller: NavHostController? = null
```

Creates a reassignable local value controller from the expression on the right.

Line 33

```
val scene = ImageComposeScene(width = 320, height = 240)
{
```

Stores a value/reference named scene from the expression on the right.

Line 34

```
CompositionLocalProvider(
```

Supplies test environment objects to the Compose navigation content below.

Line 35

```
LocalLifecycleOwner provides owner,
```

Test-only: supplies the lifecycle/state-store owner that a real app window normally provides.

Line 36

```
LocalViewModelStoreOwner provides owner
```

Test-only: supplies the lifecycle/state-store owner that a real app window normally provides.

Line 37

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 38

```
val navController = rememberNavController()
```

Stores a value/reference named navController from the expression on the right.

Line 39

```
controller = navController
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 40

```
NavHost(navController, startDestination = "input") {
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 41

```
composable("input") {}
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 42

```
composable("result/{word}") {}
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 43

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 44

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 45

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 47

```
scene.render().close()
```

Test-only: renders the Compose scene to exercise navigation startup, then releases the produced image.

Line 48

```
val navController = requireNotNull(controller)
```

Stores a value/reference named navController from the expression on the right.

Line 49

```
navController.navigate("result/hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 50

```
scene.render().close()
```

Test-only: renders the Compose scene to exercise navigation startup, then releases the produced image.

Line 51

```
assertEquals("result/{word}",
navController.currentDestination?.route)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 52

```
assertEquals("hello",
navController.currentBackStackEntry?.arguments?.read {
getString("word") })
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 53

```
navController.popBackStack()
```

Test-only: goes back to the previous destination, exercising entry cleanup/restoration.

Line 54

```
scene.render().close()
```

Test-only: renders the Compose scene to exercise navigation startup, then releases the produced image.

Line 55

```
assertEquals("input",
navController.currentDestination?.route)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 56

```
} finally {
```

Runs cleanup whether the attempt succeeded or failed.

Line 57

```
scene.close()
```

Releases the rendered test scene and cancels its Compose effects.

Line 58

```
owner.lifecycle.currentState = Lifecycle.State.DESTROYED
```

Sets this test owner’s lifecycle state so navigation can create and dispose entries correctly.

Line 59

```
owner.viewModelStore.clear()
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 60

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 61

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 62

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 63

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

34. gradle.properties

gradle.properties

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
#Kotlin
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 2

```
kotlin.code.style=official
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 3

```
kotlin.daemon.jvmargs=-Xmx3072M
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 4

```
#Gradle
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 5

```
org.gradle.jvmargs=-Xmx4096M -Dfile.encoding=UTF-8
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 6

```
org.gradle.configuration-cache=true
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 7

```
org.gradle.caching=true
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 8

```
#Android
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 9

```
android.nonTransitiveRClass=true
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 10

```
android.useAndroidX=true
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

35. libs.versions.toml

gradle/libs.versions.toml

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
[versions]
```

Starts the versions catalog section.

Line 2

```
agp = "9.1.1"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 3

```
android-compileSdk = "37"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 4

```
android-minSdk = "24"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 5

```
android-targetSdk = "37"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 6

```
androidx-activity = "1.13.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 7

```
androidx-appcompat = "1.8.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 8

```
androidx-core = "1.19.1"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 9

```
androidx-espresso = "3.7.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 10

```
androidx-lifecycle = "2.11.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 11

```
androidx-navigation = "2.10.0-rc01"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 12

```
androidx-testExt = "1.3.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 13

```
composeMultiplatform = "1.12.1"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 14

```
junit = "4.13.2"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 15

```
kotlin = "2.4.20"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 16

```
kotlin-wrappers = "2026.9.3"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 17

```
kotlinx-coroutines = "1.11.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 18

```
material3 = "1.12.0-alpha03"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 19

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 20

```
room = "3.0.3"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 21

```
sqlite = "2.7.1"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 22

```
datastore = "1.3.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 23

```
ksp = "2.3.12"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 24

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 25

```
[libraries]
```

Starts the libraries catalog section.

Line 26

```
androidx-navigation-compose = { module =
"org.jetbrains.androidx.navigation:navigation-compose",
version.ref = "androidx-navigation" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 27

```
androidx-room-runtime = { module =
"androidx.room3:room3-runtime", version.ref = "room" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 28

```
androidx-room-compiler = { module =
"androidx.room3:room3-compiler", version.ref = "room" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 29

```
androidx-sqlite-bundled = { module =
"androidx.sqlite:sqlite-bundled", version.ref = "sqlite"
}
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 30

```
androidx-sqlite-web = { module =
"androidx.sqlite:sqlite-web", version.ref = "sqlite" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 31

```
androidx-datastore = { module =
"androidx.datastore:datastore-preferences-core",
version.ref = "datastore" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 32

```
kotlin-test = { module =
"org.jetbrains.kotlin:kotlin-test", version.ref =
"kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 33

```
kotlin-testJunit = { module =
"org.jetbrains.kotlin:kotlin-test-junit", version.ref =
"kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 34

```
junit = { module = "junit:junit", version.ref = "junit" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 35

```
androidx-core-ktx = { module = "androidx.core:core-ktx",
version.ref = "androidx-core" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 36

```
androidx-testExt-junit = { module =
"androidx.test.ext:junit", version.ref =
"androidx-testExt" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 37

```
androidx-espresso-core = { module =
"androidx.test.espresso:espresso-core", version.ref =
"androidx-espresso" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 38

```
androidx-appcompat = { module =
"androidx.appcompat:appcompat", version.ref =
"androidx-appcompat" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 39

```
androidx-activity-compose = { module =
"androidx.activity:activity-compose", version.ref =
"androidx-activity" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 40

```
compose-uiTooling = { module =
"org.jetbrains.compose.ui:ui-tooling", version.ref =
"composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 41

```
androidx-lifecycle-viewmodelCompose = { module = "org.jet
brains.androidx.lifecycle:lifecycle-viewmodel-compose",
version.ref = "androidx-lifecycle" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 42

```
androidx-lifecycle-runtimeCompose = { module = "org.jetbr
ains.androidx.lifecycle:lifecycle-runtime-compose",
version.ref = "androidx-lifecycle" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 43

```
compose-runtime = { module =
"org.jetbrains.compose.runtime:runtime", version.ref =
"composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 44

```
compose-foundation = { module =
"org.jetbrains.compose.foundation:foundation",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 45

```
compose-material3 = { module =
"org.jetbrains.compose.material3:material3", version.ref
= "material3" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 46

```
compose-ui = { module = "org.jetbrains.compose.ui:ui",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 47

```
compose-components-resources = { module =
"org.jetbrains.compose.components:components-resources",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 48

```
compose-uiToolingPreview = { module =
"org.jetbrains.compose.ui:ui-tooling-preview",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 49

```
kotlinx-coroutinesSwing = { module =
"org.jetbrains.kotlinx:kotlinx-coroutines-swing",
version.ref = "kotlinx-coroutines" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 50

```
wrappers-browser = { module =
"org.jetbrains.kotlin-wrappers:kotlin-browser",
version.ref = "kotlin-wrappers" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 51

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 52

```
[plugins]
```

Starts the plugins catalog section.

Line 53

```
androidApplication = { id = "com.android.application",
version.ref = "agp" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 54

```
androidMultiplatformLibrary = { id =
"com.android.kotlin.multiplatform.library", version.ref =
 "agp" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 55

```
composeMultiplatform = { id = "org.jetbrains.compose",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 56

```
composeCompiler = { id =
"org.jetbrains.kotlin.plugin.compose", version.ref =
"kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 57

```
kotlinJvm = { id = "org.jetbrains.kotlin.jvm",
version.ref = "kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 58

```
kotlinMultiplatform = { id =
"org.jetbrains.kotlin.multiplatform", version.ref =
"kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 59

```
kotlinSerialization = { id =
"org.jetbrains.kotlin.plugin.serialization", version.ref
= "kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 60

```
ksp = { id = "com.google.devtools.ksp", version.ref =
"ksp" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 61

```
room = { id = "androidx.room3", version.ref = "room" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

36. Config.xcconfig

iosApp/Configuration/Config.xcconfig

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
TEAM_ID=
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
PRODUCT_NAME=dictionary
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 4

```
PRODUCT_BUNDLE_IDENTIFIER=com.example.dictionary.dictiona
ry$(TEAM_ID)
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
CURRENT_PROJECT_VERSION=1
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 7

```
MARKETING_VERSION=1.0
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

37. ContentView.swift

iosApp/iosApp/ContentView.swift

The native SwiftUI wrapper that opens the shared Kotlin Compose UI on iOS.

Line 1

```
import UIKit
```

Imports UIKit so this Swift wrapper can use its native UI or shared Kotlin declarations.

Line 2

```
import SwiftUI
```

Imports SwiftUI so this Swift wrapper can use its native UI or shared Kotlin declarations.

Line 3

```
import Shared
```

Imports Shared so this Swift wrapper can use its native UI or shared Kotlin declarations.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
struct ComposeView: UIViewControllerRepresentable {
```

Defines ComposeView as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 6

```
func makeUIViewController(context: Self.Context) ->
UIViewController {
```

Defines makeUIViewController(). Inputs are inside parentheses; the return type describes its answer.

Line 7

```
MainViewControllerKt.MainViewController()
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
func updateUIViewController(_ uiViewController:
UIViewController, context: Self.Context) {}
```

Defines updateUIViewController(). Inputs are inside parentheses; the return type describes its answer.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 12

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 13

```
struct ContentView: View {
```

Defines ContentView as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 14

```
var body: some View {
```

Declares the body value and its type for this object or operation. A question mark allows null; a default is used if no value is supplied.

Line 15

```
ComposeView()
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 16

```
.ignoresSafeArea()
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 17

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

38. Info.plist

iosApp/iosApp/Info.plist

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
<?xml version="1.0" encoding="UTF-8"?>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 3

```
<plist version="1.0">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 4

```
<dict>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 5

```
<key>CADisableMinimumFrameDurationOnPhone</key>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 6

```
<true/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 7

```
</dict>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 8

```
</plist>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

39. iOSApp.swift

iosApp/iosApp/iOSApp.swift

The native SwiftUI wrapper that opens the shared Kotlin Compose UI on iOS.

Line 1

```
import SwiftUI
```

Imports SwiftUI so this Swift wrapper can use its native UI or shared Kotlin declarations.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
@main
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 4

```
struct iOSApp: App {
```

Defines iOSApp as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 5

```
var body: some Scene {
```

Declares the body value and its type for this object or operation. A question mark allows null; a default is used if no value is supplied.

Line 6

```
WindowGroup {
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 7

```
ContentView()
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 9

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

40. settings.gradle.kts

settings.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
rootProject.name = "dictionary"
```

Gradle setup: configures rootProject.name = "dictionary" for the surrounding module/target. This runs during building, not when a user searches.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
pluginManagement {
```

Gradle setup: configures pluginManagement for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
repositories {
```

Gradle setup: configures repositories for the surrounding module/target. This runs during building, not when a user searches.

Line 5

```
google {
```

Gradle setup: configures google for the surrounding module/target. This runs during building, not when a user searches.

Line 6

```
mavenContent {
```

Gradle setup: configures mavenContent for the surrounding module/target. This runs during building, not when a user searches.

Line 7

```
includeGroupAndSubgroups("androidx")
```

Gradle setup: configures includeGroupAndSubgroups("androidx") for the surrounding module/target. This runs during building, not when a user searches.

Line 8

```
includeGroupAndSubgroups("com.android")
```

Gradle setup: configures includeGroupAndSubgroups("com.android") for the surrounding module/target. This runs during building, not when a user searches.

Line 9

```
includeGroupAndSubgroups("com.google")
```

Gradle setup: configures includeGroupAndSubgroups("com.google") for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 12

```
mavenCentral()
```

Gradle setup: configures mavenCentral() for the surrounding module/target. This runs during building, not when a user searches.

Line 13

```
gradlePluginPortal()
```

Gradle setup: configures gradlePluginPortal() for the surrounding module/target. This runs during building, not when a user searches.

Line 14

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 15

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 16

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 17

```
dependencyResolutionManagement {
```

Gradle setup: configures dependencyResolutionManagement for the surrounding module/target. This runs during building, not when a user searches.

Line 18

```
repositories {
```

Gradle setup: configures repositories for the surrounding module/target. This runs during building, not when a user searches.

Line 19

```
google {
```

Gradle setup: configures google for the surrounding module/target. This runs during building, not when a user searches.

Line 20

```
mavenContent {
```

Gradle setup: configures mavenContent for the surrounding module/target. This runs during building, not when a user searches.

Line 21

```
includeGroupAndSubgroups("androidx")
```

Gradle setup: configures includeGroupAndSubgroups("androidx") for the surrounding module/target. This runs during building, not when a user searches.

Line 22

```
includeGroupAndSubgroups("com.android")
```

Gradle setup: configures includeGroupAndSubgroups("com.android") for the surrounding module/target. This runs during building, not when a user searches.

Line 23

```
includeGroupAndSubgroups("com.google")
```

Gradle setup: configures includeGroupAndSubgroups("com.google") for the surrounding module/target. This runs during building, not when a user searches.

Line 24

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 25

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 26

```
mavenCentral()
```

Gradle setup: configures mavenCentral() for the surrounding module/target. This runs during building, not when a user searches.

Line 27

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 28

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 29

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 30

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 31

```
id("org.gradle.toolchains.foojay-resolver-convention")
version "1.0.0"
```

Gradle setup: configures id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0" for the surrounding module/target. This runs during building, not when a user searches.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 33

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 34

```
include(":androidApp")
```

Includes these project modules in the Gradle build.

Line 35

```
include(":desktopApp")
```

Includes these project modules in the Gradle build.

Line 36

```
include(":shared")
```

Includes these project modules in the Gradle build.

Line 37

```
include(":webApp")
```

Includes these project modules in the Gradle build.

Line 38

```
include(":browserStorage")
```

Includes these project modules in the Gradle build.

41. build.gradle.kts

shared/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.kotlin.gradle.ExperimentalWasmDsl
```

Makes ExperimentalWasmDsl available here from org.jetbrains.kotlin.gradle.

Line 2

```
import org.jetbrains.kotlin.gradle.dsl.JvmTarget
```

Makes JvmTarget available here from org.jetbrains.kotlin.gradle.dsl.

Line 3

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 4

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 5

```
alias(libs.plugins.kotlinMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
alias(libs.plugins.androidMultiplatformLibrary)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 7

```
alias(libs.plugins.composeMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 8

```
alias(libs.plugins.composeCompiler)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
alias(libs.plugins.kotlinSerialization)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 11

```
alias(libs.plugins.ksp)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 12

```
alias(libs.plugins.room)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 13

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
kotlin {
```

Gradle setup: configures kotlin for the surrounding module/target. This runs during building, not when a user searches.

Line 16

```
listOf(
```

Gradle setup: configures listOf(for the surrounding module/target. This runs during building, not when a user searches.

Line 17

```
iosArm64(),
```

Gradle setup: configures iosArm64(), for the surrounding module/target. This runs during building, not when a user searches.

Line 18

```
iosSimulatorArm64()
```

Gradle setup: configures iosSimulatorArm64() for the surrounding module/target. This runs during building, not when a user searches.

Line 19

```
).forEach { iosTarget ->
```

Gradle setup: configures).forEach { iosTarget -> for the surrounding module/target. This runs during building, not when a user searches.

Line 20

```
iosTarget.binaries.framework {
```

Gradle setup: configures iosTarget.binaries.framework for the surrounding module/target. This runs during building, not when a user searches.

Line 21

```
baseName = "Shared"
```

Gradle setup: configures baseName = "Shared" for the surrounding module/target. This runs during building, not when a user searches.

Line 22

```
isStatic = true
```

Gradle setup: configures isStatic = true for the surrounding module/target. This runs during building, not when a user searches.

Line 23

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 24

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 25

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 26

```
jvm()
```

Gradle setup: configures jvm() for the surrounding module/target. This runs during building, not when a user searches.

Line 27

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 28

```
js {
```

Gradle setup: configures js for the surrounding module/target. This runs during building, not when a user searches.

Line 29

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 30

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 31

```
binaries.executable()
```

Gradle setup: configures binaries.executable() for the surrounding module/target. This runs during building, not when a user searches.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 33

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 34

```
@OptIn(ExperimentalWasmDsl::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 35

```
wasmJs {
```

Gradle setup: configures wasmJs for the surrounding module/target. This runs during building, not when a user searches.

Line 36

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 37

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 38

```
binaries.executable()
```

Gradle setup: configures binaries.executable() for the surrounding module/target. This runs during building, not when a user searches.

Line 39

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 40

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 41

```
android {
```

Gradle setup: configures android for the surrounding module/target. This runs during building, not when a user searches.

Line 42

```
namespace = "com.example.dictionary.shared"
```

Gradle setup: configures namespace = "com.example.dictionary.shared" for the surrounding module/target. This runs during building, not when a user searches.

Line 43

```
compileSdk =
libs.versions.android.compileSdk.get().toInt()
```

Gradle setup: configures compileSdk = libs.versions.android.compileSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 44

```
minSdk = libs.versions.android.minSdk.get().toInt()
```

Gradle setup: configures minSdk = libs.versions.android.minSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 45

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 46

```
compilerOptions {
```

Gradle setup: configures compilerOptions for the surrounding module/target. This runs during building, not when a user searches.

Line 47

```
jvmTarget = JvmTarget.JVM_11
```

Gradle setup: configures jvmTarget = JvmTarget.JVM_11 for the surrounding module/target. This runs during building, not when a user searches.

Line 48

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 49

```
androidResources {
```

Gradle setup: configures androidResources for the surrounding module/target. This runs during building, not when a user searches.

Line 50

```
enable = true
```

Gradle setup: configures enable = true for the surrounding module/target. This runs during building, not when a user searches.

Line 51

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 52

```
withHostTest {
```

Gradle setup: configures withHostTest for the surrounding module/target. This runs during building, not when a user searches.

Line 53

```
isIncludeAndroidResources = true
```

Gradle setup: configures isIncludeAndroidResources = true for the surrounding module/target. This runs during building, not when a user searches.

Line 54

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 55

```
withDeviceTestBuilder {
```

Gradle setup: configures withDeviceTestBuilder for the surrounding module/target. This runs during building, not when a user searches.

Line 56

```
sourceSetTreeName = "test"
```

Gradle setup: configures sourceSetTreeName = "test" for the surrounding module/target. This runs during building, not when a user searches.

Line 57

```
}.configure {
```

Gradle setup: configures }.configure for the surrounding module/target. This runs during building, not when a user searches.

Line 58

```
instrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
```

Gradle setup: configures instrumentationRunner = "androidx.test.runner.AndroidJUnitRunner" for the surrounding module/target. This runs during building, not when a user searches.

Line 59

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 60

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 61

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 62

```
sourceSets {
```

Gradle setup: configures sourceSets for the surrounding module/target. This runs during building, not when a user searches.

Line 63

```
androidMain.dependencies {
```

Starts dependencies for androidMain. This determines which targets can see the following libraries.

Line 64

```
implementation(libs.androidx.sqlite.bundled)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 65

```
implementation("io.ktor:ktor-client-okhttp:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 66

```
implementation(libs.compose.uiToolingPreview)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 67

```
implementation(libs.compose.uiTooling)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 68

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 69

```
commonMain.dependencies {
```

Starts dependencies for commonMain. This determines which targets can see the following libraries.

Line 70

```
implementation(libs.compose.runtime)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 71

```
implementation(libs.compose.foundation)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 72

```
implementation(libs.compose.material3)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 73

```
implementation(libs.compose.ui)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 74

```
implementation(libs.compose.components.resources)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 75

```
implementation(libs.compose.uiToolingPreview)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 76

```
implementation(libs.androidx.lifecycle.viewmodelCompose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 77

```
implementation(libs.androidx.lifecycle.runtimeCompose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 78

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 79

```
// Modules 6 and 9: shared networking and JSON
dependencies.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 80

```
implementation("io.ktor:ktor-client-logging:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 81

```
implementation(libs.androidx.room.runtime)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 82

```
implementation(libs.androidx.datastore)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 83

```
implementation("io.ktor:ktor-client-core:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 84

```
implementation("io.ktor:ktor-client-content-negotiation:3
.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 85

```
implementation("io.ktor:ktor-serialization-kotlinx-json:3
.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 86

```
implementation("org.jetbrains.kotlinx:kotlinx-serializati
on-json:1.7.3")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 87

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-
core:1.11.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 88

```
implementation(libs.androidx.navigation.compose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 89

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 90

```
commonTest.dependencies {
```

Starts dependencies for commonTest. This determines which targets can see the following libraries.

Line 91

```
implementation(libs.kotlin.test)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 92

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 93

```
iosMain.dependencies {
```

Starts dependencies for iosMain. This determines which targets can see the following libraries.

Line 94

```
implementation(libs.androidx.sqlite.bundled)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 95

```
implementation("io.ktor:ktor-client-darwin:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 96

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 97

```
jvmTest.dependencies {
```

Starts dependencies for jvmTest. This determines which targets can see the following libraries.

Line 98

```
implementation("io.ktor:ktor-client-mock:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 99

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 100

```
jvmMain.dependencies {
```

Starts dependencies for jvmMain. This determines which targets can see the following libraries.

Line 101

```
implementation(libs.androidx.sqlite.bundled)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 102

```
implementation("io.ktor:ktor-client-cio:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 103

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 104

```
jsMain.dependencies {
```

Starts dependencies for jsMain. This determines which targets can see the following libraries.

Line 105

```
implementation(libs.wrappers.browser)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 106

```
implementation("io.ktor:ktor-client-js:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 107

```
implementation(project(":browserStorage"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 108

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 109

```
wasmJsMain.dependencies {
```

Starts dependencies for wasmJsMain. This determines which targets can see the following libraries.

Line 110

```
implementation("io.ktor:ktor-client-js:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 111

```
implementation(project(":browserStorage"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 112

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 113

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 114

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 115

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 116

```
dependencies {
```

Gradle setup: configures dependencies for the surrounding module/target. This runs during building, not when a user searches.

Line 117

```
androidRuntimeClasspath(libs.compose.uiTooling)
```

Gradle setup: configures androidRuntimeClasspath(libs.compose.uiTooling) for the surrounding module/target. This runs during building, not when a user searches.

Line 118

```
listOf("kspAndroid", "kspJvm", "kspIosArm64",
"kspIosSimulatorArm64", "kspJs", "kspWasmJs").forEach {
```

Gradle setup: configures listOf("kspAndroid", "kspJvm", "kspIosArm64", "kspIosSimulatorArm64", "kspJs", "kspWasmJs").forEach for the surrounding module/target. This runs during building, not when a user searches.

Line 119

```
add(it, libs.androidx.room.compiler)
```

Adds the Room code generator to the current platform's KSP configuration.

Line 120

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 121

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 123

```
room3 {
```

Gradle setup: configures room3 for the surrounding module/target. This runs during building, not when a user searches.

Line 124

```
schemaDirectory(layout.projectDirectory.dir("schemas"))
```

Records Room's database schema snapshots in this directory for checking/version history.

Line 125

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

42. AppStorage.kt (androidMain)

shared/src/androidMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import android.content.Context
```

Makes Context available here from android.content.

Line 4

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 5

```
import androidx.datastore.core.FileStorage
```

Makes FileStorage available here from androidx.datastore.core.

Line 6

```
import androidx.datastore.preferences.core.PreferencesFil
eSerializer
```

Makes PreferencesFileSerializer available here from androidx.datastore.preferences.core.

Line 7

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 8

```
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
```

Makes BundledSQLiteDriver available here from androidx.sqlite.driver.bundled.

Line 9

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 10

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 11

```
private var appStorage: AppStorage? = null
```

Retains Android’s shared storage instance for reuse, including Activity recreation.

Line 12

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 13

```
fun createAppStorage(context: Context): AppStorage {
```

Defines createAppStorage(). Inputs are inside parentheses; the return type describes its answer.

Line 14

```
appStorage?.let { return it }
```

If storage already exists, return it instead of opening another DataStore for the same file.

Line 15

```
val appContext = context.applicationContext
```

Uses Android’s application-lifetime context rather than retaining an Activity object.

Line 16

```
val database = Room.databaseBuilder<DictionaryDatabase>(
```

Starts building a Room database using this platform’s file location or browser driver.

Line 17

```
context = appContext,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 18

```
name =
appContext.getDatabasePath("dictionary.db").absolutePath
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 19

```
).setDriver(BundledSQLiteDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 20

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 21

```
storage = FileStorage(
```

Uses JVM/Android file storage for the preference data.

Line 22

```
serializer = PreferencesFileSerializer,
```

Chooses the preference serializer that can read/write DataStore’s preference format.

Line 23

```
produceFile = {
appContext.filesDir.resolve("dictionary.preferences_pb")
}
```

Provides the real file/path where this platform stores preferences.

Line 24

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 25

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 26

```
val storage = AppStorage(database, preferences)
```

Creates/retains the platform's persistent storage objects before passing them into the shared App.

Line 27

```
appStorage = storage
```

Keeps the new Android storage instance so later calls reuse it.

Line 28

```
return storage
```

Sends the following value/expression back as this function's answer: storage.

Line 29

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

43. HttpClientFactory.android.kt (androidMain)

shared/src/androidMain/kotlin/com/example/dictionary/data/HttpClientFactory.android.kt

Creates this platform's Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.okhttp.OkHttp
```

Makes OkHttp available here from io.ktor.client.engine.okhttp.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform's implementation for the shared expect declaration.

Line 7

```
return HttpClient(OkHttp) { configureDictionaryClient() }
```

Creates this platform's HTTP client using OkHttp and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

44. MainViewController.kt (iosMain)

shared/src/iosMain/kotlin/com/example/dictionary/MainViewController.kt

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.ui.window.ComposeUIViewController
```

Makes ComposeUIViewController available here from androidx.compose.ui.window.

Line 4

```
import com.example.dictionary.data.createAppStorage
```

Makes createAppStorage available here from com.example.dictionary.data.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
private val storage by lazy { createAppStorage() }
```

Creates/retains the platform’s persistent storage objects before passing them into the shared App.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
fun MainViewController() = ComposeUIViewController {
App(storage) }
```

Displays the shared Compose App and supplies the storage built by this platform.

45. AppStorage.kt (iosMain)

shared/src/iosMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform’s persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.okio.OkioStorage
```

Makes OkioStorage available here from androidx.datastore.core.okio.

Line 5

```
import
androidx.datastore.preferences.core.PreferencesSerializer
```

Makes PreferencesSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
```

Makes BundledSQLiteDriver available here from androidx.sqlite.driver.bundled.

Line 8

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 9

```
import kotlinx.cinterop.ExperimentalForeignApi
```

Makes ExperimentalForeignApi available here from kotlinx.cinterop.

Line 10

```
import okio.FileSystem
```

Makes FileSystem available here from okio.

Line 11

```
import okio.Path.Companion.toPath
```

Makes toPath available here from okio.Path.Companion.

Line 12

```
import platform.Foundation.NSDocumentDirectory
```

Makes NSDocumentDirectory available here from platform.Foundation.

Line 13

```
import platform.Foundation.NSFileManager
```

Makes NSFileManager available here from platform.Foundation.

Line 14

```
import platform.Foundation.NSUserDomainMask
```

Makes NSUserDomainMask available here from platform.Foundation.

Line 15

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 16

```
@OptIn(ExperimentalForeignApi::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 17

```
fun createAppStorage(): AppStorage {
```

Defines createAppStorage(). Inputs are inside parentheses; the return type describes its answer.

Line 18

```
val directory =
NSFileManager.defaultManager.URLForDirectory(
```

Asks iOS for the platform directory used to keep the app’s documents.

Line 19

```
NSDocumentDirectory, NSUserDomainMask, null, true, null
```

Selects the iOS Documents directory, in the user domain, creating it if needed.

Line 20

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
val path = requireNotNull(directory?.path)
```

Requires a valid storage directory path before constructing the database/preferences.

Line 22

```
val database =
Room.databaseBuilder<DictionaryDatabase>(name =
"$path/dictionary.db")
```

Starts building a Room database using this platform’s file location or browser driver.

Line 23

```
.setDriver(BundledSQLiteDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 24

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 25

```
storage = OkioStorage(
```

Uses the cross-platform file interface for iOS preference storage.

Line 26

```
fileSystem = FileSystem.SYSTEM,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 27

```
serializer = PreferencesSerializer,
```

Chooses the preference serializer that can read/write DataStore's preference format.

Line 28

```
producePath = {
"$path/dictionary.preferences_pb".toPath() }
```

Provides the real file/path where this platform stores preferences.

Line 29

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 30

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 31

```
return AppStorage(database, preferences)
```

Returns the database and DataStore together so the platform entry point can pass them to App.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

46. HttpClientFactory.ios.kt (iosMain)

shared/src/iosMain/kotlin/com/example/dictionary/data/HttpClientFactory.ios.kt

Creates this platform’s Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.darwin.Darwin
```

Makes Darwin available here from io.ktor.client.engine.darwin.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform’s implementation for the shared expect declaration.

Line 7

```
return HttpClient(Darwin) { configureDictionaryClient() }
```

Creates this platform’s HTTP client using Darwin and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

47. AppStorage.kt (jsMain)

shared/src/jsMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.okio.WebLocalStorage
```

Makes WebLocalStorage available here from androidx.datastore.core.okio.

Line 5

```
import
androidx.datastore.preferences.core.PreferencesSerializer
```

Makes PreferencesSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 8

```
import com.example.dictionary.browser.createBrowserDriver
```

Makes createBrowserDriver available here from com.example.dictionary.browser.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
fun createAppStorage(): AppStorage {
```

Defines createAppStorage(). Inputs are inside parentheses; the return type describes its answer.

Line 11

```
val database =
Room.databaseBuilder<DictionaryDatabase>(name =
"dictionary.db")
```

Starts building a Room database using this platform’s file location or browser driver.

Line 12

```
.setDriver(createBrowserDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 13

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 14

```
storage = WebLocalStorage(
```

Uses the browser’s localStorage for the small preference data.

Line 15

```
serializer = PreferencesSerializer,
```

Chooses the preference serializer that can read/write DataStore’s preference format.

Line 16

```
name = "dictionary.preferences"
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 17

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 19

```
return AppStorage(database, preferences)
```

Returns the database and DataStore together so the platform entry point can pass them to App.

Line 20

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

48. HttpClientFactory.js.kt (jsMain)

shared/src/jsMain/kotlin/com/example/dictionary/data/HttpClientFactory.js.kt

Creates this platform's Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.js.Js
```

Makes Js available here from io.ktor.client.engine.js.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform's implementation for the shared expect declaration.

Line 7

```
return HttpClient(Js) { configureDictionaryClient() }
```

Creates this platform's HTTP client using Js and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

49. AppStorage.kt (jvmMain)

shared/src/jvmMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.FileStorage
```

Makes FileStorage available here from androidx.datastore.core.

Line 5

```
import androidx.datastore.preferences.core.PreferencesFil
eSerializer
```

Makes PreferencesFileSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
```

Makes BundledSQLiteDriver available here from androidx.sqlite.driver.bundled.

Line 8

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 9

```
import java.io.File
```

Makes File available here from java.io.

Line 10

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 11

```
fun createAppStorage(directory: File =
File(System.getProperty("user.home"),
".kmp-dictionary")): AppStorage {
```

Defaults desktop storage to the current user's home folder under .kmp-dictionary.

Line 12

```
directory.mkdirs()
```

Creates the desktop storage directory if it does not already exist.

Line 13

```
val database = Room.databaseBuilder<DictionaryDatabase>(
```

Starts building a Room database using this platform's file location or browser driver.

Line 14

```
name = File(directory, "dictionary.db").absolutePath
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 15

```
).setDriver(BundledSQLiteDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 16

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 17

```
storage = FileStorage(
```

Uses JVM/Android file storage for the preference data.

Line 18

```
serializer = PreferencesFileSerializer,
```

Chooses the preference serializer that can read/write DataStore's preference format.

Line 19

```
produceFile = { File(directory,
"dictionary.preferences_pb") }
```

Provides the real file/path where this platform stores preferences.

Line 20

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 22

```
return AppStorage(database, preferences)
```

Returns the database and DataStore together so the platform entry point can pass them to App.

Line 23

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

50. HttpClientFactory.jvm.kt (jvmMain)

shared/src/jvmMain/kotlin/com/example/dictionary/data/HttpClientFactory.jvm.kt

Creates this platform's Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.cio.CIO
```

Makes CIO available here from io.ktor.client.engine.cio.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform's implementation for the shared expect declaration.

Line 7

```
return HttpClient(CIO) { configureDictionaryClient() }
```

Creates this platform's HTTP client using CIO and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

51. PersistenceTest.kt (jvmTest)

shared/src/jvmTest/kotlin/com/example/dictionary/PersistenceTest.kt

Automated checks with assertions. Study after the app logic; this code runs during tests, not normal searches.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.FileStorage
```

Makes FileStorage available here from androidx.datastore.core.

Line 5

```
import androidx.datastore.preferences.core.PreferencesFil
eSerializer
```

Makes PreferencesFileSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
```

Makes BundledSQLiteDriver available here from androidx.sqlite.driver.bundled.

Line 8

```
import com.example.dictionary.data.DictionaryPreferences
```

Makes DictionaryPreferences available here from com.example.dictionary.data.

Line 9

```
import
com.example.dictionary.data.DictionaryRepositoryImpl
```

Makes DictionaryRepositoryImpl available here from com.example.dictionary.data.

Line 10

```
import
com.example.dictionary.data.configureDictionaryClient
```

Makes configureDictionaryClient available here from com.example.dictionary.data.

Line 11

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 12

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 13

```
import io.ktor.client.engine.mock.MockEngine
```

Makes MockEngine available here from io.ktor.client.engine.mock.

Line 14

```
import io.ktor.client.engine.mock.respond
```

Makes respond available here from io.ktor.client.engine.mock.

Line 15

```
import io.ktor.http.HttpHeaders
```

Makes HttpHeaders available here from io.ktor.http.

Line 16

```
import io.ktor.http.HttpStatusCode
```

Makes HttpStatusCode available here from io.ktor.http.

Line 17

```
import io.ktor.http.headersOf
```

Makes headersOf available here from io.ktor.http.

Line 18

```
import java.io.File
```

Makes File available here from java.io.

Line 19

```
import java.nio.file.Files
```

Makes Files available here from java.nio.file.

Line 20

```
import kotlinx.coroutines.CoroutineScope
```

Makes CoroutineScope available here from kotlinx.coroutines.

Line 21

```
import kotlinx.coroutines.Dispatchers
```

Makes Dispatchers available here from kotlinx.coroutines.

Line 22

```
import kotlinx.coroutines.SupervisorJob
```

Makes SupervisorJob available here from kotlinx.coroutines.

Line 23

```
import kotlinx.coroutines.cancelAndJoin
```

Makes cancelAndJoin available here from kotlinx.coroutines.

Line 24

```
import kotlinx.coroutines.flow.first
```

Makes first available here from kotlinx.coroutines.flow.

Line 25

```
import kotlinx.coroutines.runBlocking
```

Makes runBlocking available here from kotlinx.coroutines.

Line 26

```
import kotlinx.coroutines.withTimeout
```

Makes withTimeout available here from kotlinx.coroutines.

Line 27

```
import kotlin.test.AfterTest
```

Makes AfterTest available here from kotlin.test.

Line 28

```
import kotlin.test.Test
```

Makes Test available here from kotlin.test.

Line 29

```
import kotlin.test.assertEquals
```

Makes assertEquals available here from kotlin.test.

Line 30

```
import kotlin.test.assertFalse
```

Makes assertFalse available here from kotlin.test.

Line 31

```
import kotlin.test.assertNotNull
```

Makes assertNotNull available here from kotlin.test.

Line 32

```
import kotlin.test.assertNull
```

Makes assertNull available here from kotlin.test.

Line 33

```
import kotlin.test.assertTrue
```

Makes assertTrue available here from kotlin.test.

Line 34

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 35

```
class PersistenceTest {
```

Defines PersistenceTest as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 36

```
private val directory =
Files.createTempDirectory("dictionary-test-").toFile()
```

Stores a value/reference named directory from the expression on the right.

Line 37

```
private var database: DictionaryDatabase? = null
```

Creates a reassignable local value database from the expression on the right.

Line 38

```
private val clients = mutableListOf<HttpClient>()
```

Stores a value/reference named clients from the expression on the right. An empty collection will be filled later.

Line 39

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 40

```
private fun openDatabase(): DictionaryDatabase {
```

Defines openDatabase(). Inputs are inside parentheses; the return type describes its answer.

Line 41

```
return Room.databaseBuilder<DictionaryDatabase>(
```

Starts building a Room database using this platform’s file location or browser driver.

Line 42

```
name = File(directory, "test.db").absolutePath
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 43

```
).setDriver(BundledSQLiteDriver()).build().also {
database = it }
```

Supplies the SQLite implementation, then builds the Room database object.

Line 44

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 45

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 46

```
private fun client(
```

Defines client(). Inputs are inside parentheses; the return type describes its answer.

Line 47

```
formal: String = """[{"fl":"noun","shortdef":["a
greeting"],"extra":"ignored"}]""",
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 48

```
urban: String =
"""{"list":[{"word":"hello","definition":"a
[greeting]","example":"hello there"}]}""",
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 49

```
status: HttpStatusCode = HttpStatusCode.OK,
```

Test-helper input: choose the mocked HTTP response status, defaulting to 200 OK.

Line 50

```
onRequest: () -> Unit = {}
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 51

```
): HttpClient {
```

Finishes the test helper inputs and specifies that the function returns a configured HttpClient.

Line 52

```
return HttpClient(MockEngine { request ->
```

Creates a test HTTP client using the mocked engine below. Its responses are supplied by the test, not real online dictionary services.

Line 53

```
onRequest()
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 54

```
val body = if (request.url.host ==
"api.urbandictionary.com") urban else formal
```

Stores a value/reference named body from the expression on the right.

Line 55

```
respond(body, status, headersOf(HttpHeaders.ContentType,
"application/json"))
```

Returns this fake HTTP body/status/headers from the mocked engine.

Line 56

```
}) { configureDictionaryClient() }.also { clients.add(it)
 }
```

Closes the mock handler, applies the real shared client plugins, and retains this client in the test cleanup list.

Line 57

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 58

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 59

```
@AfterTest
```

Runs the following cleanup after each test so separate tests do not share resources/results.

Line 60

```
fun cleanUp() {
```

Defines cleanUp(). Inputs are inside parentheses; the return type describes its answer.

Line 61

```
clients.forEach { it.close() }
```

Releases the object's resources when finished.

Line 62

```
database?.close()
```

Closes the database used by this test so it can be reopened or cleaned up.

Line 63

```
directory.deleteRecursively()
```

Test cleanup: deletes the temporary test directory, not the user's normal app database.

Line 64

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 65

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 66

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 67

```
fun
definitionsAndFavoritesSurviveReopeningWithoutNetwork() =
 runBlocking {
```

Defines definitionsAndFavoritesSurviveReopeningWithoutNetwork(). Inputs are inside parentheses; the return type describes its answer.

Line 68

```
withTimeout(20000) {
```

Limits this test block's running time so a hung operation fails instead of waiting indefinitely.

Line 69

```
val db = openDatabase()
```

Stores a value/reference named db from the expression on the right.

Line 70

```
val repository =
DictionaryRepositoryImpl(db.historyDao(), client(),
"test-key")
```

Receives/keeps repository as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 71

```
val result = assertNotNull(repository.fetchDefinitions("
Hello "))
```

Test check: requires an actual value rather than null; returns that value for further checks when successful.

Line 72

```
assertEquals("a greeting",
result.formalEntries.first().definition)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 73

```
assertEquals("a greeting",
result.urbanEntries.first().definition)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 74

```
repository.toggleFavorite("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 75

```
db.close()
```

Closes the database used by this test so it can be reopened or cleaned up.

Line 76

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 77

```
val reopened = openDatabase()
```

Stores a value/reference named reopened from the expression on the right.

Line 78

```
var requests = 0
```

Creates a reassignable local value requests from the expression on the right.

Line 79

```
val offline = client(status =
HttpStatusCode.ServiceUnavailable) { requests++ }
```

Stores a value/reference named offline from the expression on the right.

Line 80

```
val restored =
DictionaryRepositoryImpl(reopened.historyDao(), offline,
"test-key")
```

Stores a value/reference named restored from the expression on the right.

Line 81

```
assertTrue(assertNotNull(restored.fetchDefinitions("hello
")).isCached)
```

Test check: requires an actual value rather than null; returns that value for further checks when successful.

Line 82

```
assertEquals(0, requests)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 83

```
val history = restored.observeHistory().first()
```

Reads the first/current emitted value here, rather than collecting changes forever.

Line 84

```
assertEquals(1, history.size)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 85

```
assertTrue(history.first().isFavorite)
```

Test check: requires this condition to be true.

Line 86

```
assertEquals("hello", history.first().word)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 87

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 88

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 89

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 90

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 91

```
fun failedRefreshKeepsTheSavedWordAndFavorite() =
runBlocking {
```

Defines failedRefreshKeepsTheSavedWordAndFavorite(). Inputs are inside parentheses; the return type describes its answer.

Line 92

```
withTimeout(20000) {
```

Limits this test block’s running time so a hung operation fails instead of waiting indefinitely.

Line 93

```
val db = openDatabase()
```

Stores a value/reference named db from the expression on the right.

Line 94

```
val original = DictionaryRepositoryImpl(db.historyDao(),
client(), "test-key")
```

Stores a value/reference named original from the expression on the right.

Line 95

```
original.fetchDefinitions("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 96

```
original.toggleFavorite("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 97

```
val offline = DictionaryRepositoryImpl(
```

Stores a value/reference named offline from the expression on the right.

Line 98

```
db.historyDao(), client(status =
HttpStatusCode.ServiceUnavailable), "test-key"
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 99

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 100

```
assertNull(offline.fetchDefinitions("hello", forceRefresh
 = true))
```

Test check: requires the operation’s answer to be null.

Line 101

```
assertEquals("a greeting", assertNotNull(offline.fetchDef
initions("hello")).formalEntries.first().definition)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 102

```
assertTrue(offline.observeHistory().first().first().isFav
orite)
```

Test check: requires this condition to be true.

Line 103

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 104

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 105

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 106

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 107

```
fun successfulRefreshUpdatesDefinitionsWithoutClearingFav
orites() = runBlocking {
```

Defines successfulRefreshUpdatesDefinitionsWithoutClearingFavorites(). Inputs are inside parentheses; the return type describes its answer.

Line 108

```
withTimeout(20000) {
```

Limits this test block’s running time so a hung operation fails instead of waiting indefinitely.

Line 109

```
val db = openDatabase()
```

Stores a value/reference named db from the expression on the right.

Line 110

```
val original = DictionaryRepositoryImpl(db.historyDao(),
client(), "test-key")
```

Stores a value/reference named original from the expression on the right.

Line 111

```
original.fetchDefinitions("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 112

```
original.toggleFavorite("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 113

```
val updated = DictionaryRepositoryImpl(db.historyDao(),
client(
```

Stores a value/reference named updated from the expression on the right.

Line 114

```
formal = """[{"fl":"noun","shortdef":["updated
greeting"]}]"""
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 115

```
), "test-key")
```

Finishes constructing the test repository with an intentionally fake API key; no live service is contacted.

Line 116

```
val result =
assertNotNull(updated.fetchDefinitions("hello",
forceRefresh = true))
```

Test check: requires an actual value rather than null; returns that value for further checks when successful.

Line 117

```
assertFalse(result.isCached)
```

Test check: requires this condition to be false.

Line 118

```
assertEquals("updated greeting",
result.formalEntries.first().definition)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 119

```
assertTrue(updated.observeHistory().first().first().isFav
orite)
```

Test check: requires this condition to be true.

Line 120

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 121

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 123

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 124

```
fun spellingOnlyAndMalformedResponsesAreNotSavedAsDefinit
ions() = runBlocking {
```

Defines spellingOnlyAndMalformedResponsesAreNotSavedAsDefinitions(). Inputs are inside parentheses; the return type describes its answer.

Line 125

```
withTimeout(20000) {
```

Limits this test block’s running time so a hung operation fails instead of waiting indefinitely.

Line 126

```
val db = openDatabase()
```

Stores a value/reference named db from the expression on the right.

Line 127

```
val spelling = DictionaryRepositoryImpl(db.historyDao(),
client(
```

Stores a value/reference named spelling from the expression on the right.

Line 128

```
formal = """["hello"]""", urban = """{"list":[]}"""
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 129

```
), "test-key")
```

Finishes constructing the test repository with an intentionally fake API key; no live service is contacted.

Line 130

```
assertEquals("hello", assertNotNull(spelling.fetchDefinit
ions("helo")).didYouMeanSuggestion)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 131

```
assertTrue(spelling.observeHistory().first().isEmpty())
```

Test check: requires this condition to be true.

Line 132

```
val broken = DictionaryRepositoryImpl(db.historyDao(),
client(
```

Stores a value/reference named broken from the expression on the right.

Line 133

```
formal = "invalid JSON", urban = "invalid JSON"
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 134

```
), "test-key")
```

Finishes constructing the test repository with an intentionally fake API key; no live service is contacted.

Line 135

```
assertNull(broken.fetchDefinitions("hello"))
```

Test check: requires the operation’s answer to be null.

Line 136

```
assertTrue(broken.observeHistory().first().isEmpty())
```

Test check: requires this condition to be true.

Line 137

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 138

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 139

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 140

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 141

```
fun preferenceSurvivesRecreatingDataStore() = runBlocking
 {
```

Defines preferenceSurvivesRecreatingDataStore(). Inputs are inside parentheses; the return type describes its answer.

Line 142

```
withTimeout(20000) {
```

Limits this test block's running time so a hung operation fails instead of waiting indefinitely.

Line 143

```
val file = File(directory, "test.preferences_pb")
```

Stores a value/reference named file from the expression on the right.

Line 144

```
val firstJob = SupervisorJob()
```

Stores a value/reference named firstJob from the expression on the right.

Line 145

```
val firstStore = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 146

```
storage = FileStorage(PreferencesFileSerializer) { file
},
```

Uses JVM/Android file storage for the preference data.

Line 147

```
scope = CoroutineScope(firstJob + Dispatchers.IO)
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 148

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 149

```
val firstPreferences = DictionaryPreferences(firstStore)
```

Stores a value/reference named firstPreferences from the expression on the right.

Line 150

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 151

```
assertFalse(firstPreferences.observeShowSlang().first())
```

Test check: requires this condition to be false.

Line 152

```
firstPreferences.saveShowSlang(true)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 153

```
} finally {
```

Runs cleanup whether the attempt succeeded or failed.

Line 154

```
firstJob.cancelAndJoin()
```

Stops this test-owned coroutine job and waits for it to end before reopening the same preference file.

Line 155

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 156

```
val secondJob = SupervisorJob()
```

Stores a value/reference named secondJob from the expression on the right.

Line 157

```
val secondStore = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 158

```
storage = FileStorage(PreferencesFileSerializer) { file
},
```

Uses JVM/Android file storage for the preference data.

Line 159

```
scope = CoroutineScope(secondJob + Dispatchers.IO)
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 160

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 161

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 162

```
assertTrue(DictionaryPreferences(secondStore).observeShow
Slang().first())
```

Test check: requires this condition to be true.

Line 163

```
} finally {
```

Runs cleanup whether the attempt succeeded or failed.

Line 164

```
secondJob.cancelAndJoin()
```

Stops this test-owned coroutine job and waits for it to end before reopening the same preference file.

Line 165

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 166

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 167

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 168

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

52. AppStorage.kt (wasmJsMain)

shared/src/wasmJsMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.okio.WebLocalStorage
```

Makes WebLocalStorage available here from androidx.datastore.core.okio.

Line 5

```
import
androidx.datastore.preferences.core.PreferencesSerializer
```

Makes PreferencesSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 8

```
import com.example.dictionary.browser.createBrowserDriver
```

Makes createBrowserDriver available here from com.example.dictionary.browser.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
fun createAppStorage(): AppStorage {
```

Defines createAppStorage(). Inputs are inside parentheses; the return type describes its answer.

Line 11

```
val database =
Room.databaseBuilder<DictionaryDatabase>(name =
"dictionary.db")
```

Starts building a Room database using this platform’s file location or browser driver.

Line 12

```
.setDriver(createBrowserDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 13

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 14

```
storage = WebLocalStorage(
```

Uses the browser’s localStorage for the small preference data.

Line 15

```
serializer = PreferencesSerializer,
```

Chooses the preference serializer that can read/write DataStore’s preference format.

Line 16

```
name = "dictionary.preferences"
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 17

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 19

```
return AppStorage(database, preferences)
```

Returns the database and DataStore together so the platform entry point can pass them to App.

Line 20

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

53. HttpClientFactory.wasmJs.kt (wasmJsMain)

shared/src/wasmJsMain/kotlin/com/example/dictionary/data/HttpClientFactory.wasmJs.kt

Creates this platform's Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.js.Js
```

Makes Js available here from io.ktor.client.engine.js.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform's implementation for the shared expect declaration.

Line 7

```
return HttpClient(Js) { configureDictionaryClient() }
```

Creates this platform's HTTP client using Js and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

54. build.gradle.kts

webApp/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.kotlin.gradle.ExperimentalWasmDsl
```

Makes ExperimentalWasmDsl available here from org.jetbrains.kotlin.gradle.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
alias(libs.plugins.kotlinMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
alias(libs.plugins.composeMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
alias(libs.plugins.composeCompiler)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 8

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 9

```
kotlin {
```

Gradle setup: configures kotlin for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
js {
```

Gradle setup: configures js for the surrounding module/target. This runs during building, not when a user searches.

Line 11

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 12

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 13

```
binaries.executable()
```

Gradle setup: configures binaries.executable() for the surrounding module/target. This runs during building, not when a user searches.

Line 14

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 15

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 16

```
@OptIn(ExperimentalWasmDsl::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 17

```
wasmJs {
```

Gradle setup: configures wasmJs for the surrounding module/target. This runs during building, not when a user searches.

Line 18

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 19

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 20

```
binaries.executable()
```

Gradle setup: configures binaries.executable() for the surrounding module/target. This runs during building, not when a user searches.

Line 21

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 22

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 23

```
sourceSets {
```

Gradle setup: configures sourceSets for the surrounding module/target. This runs during building, not when a user searches.

Line 24

```
commonMain.dependencies {
```

Starts dependencies for commonMain. This determines which targets can see the following libraries.

Line 25

```
implementation(project(":shared"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 26

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 27

```
implementation(libs.compose.ui)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 28

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 29

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 30

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

55. main.kt (webMain)

webApp/src/webMain/kotlin/com/example/dictionary/main.kt

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.data.createAppStorage
```

Makes createAppStorage available here from com.example.dictionary.data.

Line 4

```
import androidx.compose.ui.ExperimentalComposeUiApi
```

Makes ExperimentalComposeUiApi available here from androidx.compose.ui.

Line 5

```
import androidx.compose.ui.window.ComposeViewport
```

Makes ComposeViewport available here from androidx.compose.ui.window.

Line 6

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 7

```
@OptIn(ExperimentalComposeUiApi::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 8

```
fun main() {
```

Defines main(). Inputs are inside parentheses; the return type describes its answer.

Line 9

```
val storage = createAppStorage()
```

Creates/retains the platform’s persistent storage objects before passing them into the shared App.

Line 10

```
ComposeViewport {
```

Mounts the shared Compose UI into the browser page.

Line 11

```
App(storage)
```

Displays the shared Compose App and supplies the storage built by this platform.

Line 12

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 13

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

56. index.html (webMain)

webApp/src/webMain/resources/index.html

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
<!DOCTYPE html>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
<html lang="en">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 3

```
<head>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 4

```
<meta charset="UTF-8">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 5

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 6

```
<title>dictionary</title>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 7

```
<link type="text/css" rel="stylesheet" href="styles.css">
```

Loads the page stylesheet.

Line 8

```
</head>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 9

```
<body style="text-align: center; align-content: center">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 10

```
<svg xmlns="http://www.w3.org/2000/svg" width="40"
height="40" viewBox="0 0 50 50" role="presentation">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 11

```
<circle cx="25" cy="25" r="20" stroke="#ccc"
stroke-width="4" fill="none"/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 12

```
<circle cx="25" cy="25" r="20" stroke="#333"
stroke-width="4" fill="none" stroke-linecap="round"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 13

```
stroke-dasharray="90 125">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 14

```
<animateTransform attributeName="transform" type="rotate"
 from="0 25 25" to="360 25 25" dur="1s"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 15

```
repeatCount="indefinite"/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 16

```
</circle>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 17

```
</svg>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 18

```
<script type="application/javascript"
src="webApp.js"></script>
```

Loads the compiled browser application JavaScript bundle.

Line 19

```
</body>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 20

```
</html>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

57. styles.css (webMain)

webApp/src/webMain/resources/styles.css

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
html, body {
```

Browser layout/build setting: html, body {; sizes the page or configures the browser development environment.

Line 2

```
width: 100%;
```

Browser layout/build setting: width: 100%;; sizes the page or configures the browser development environment.

Line 3

```
height: 100%;
```

Browser layout/build setting: height: 100%;; sizes the page or configures the browser development environment.

Line 4

```
margin: 0;
```

Browser layout/build setting: margin: 0;; sizes the page or configures the browser development environment.

Line 5

```
padding: 0;
```

Browser layout/build setting: padding: 0;; sizes the page or configures the browser development environment.

Line 6

```
overflow: hidden;
```

Browser layout/build setting: overflow: hidden;; sizes the page or configures the browser development environment.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

58. storage.js

webApp/webpack.config.d/storage.js

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
// SQLite OPFS storage needs a secure,
cross-origin-isolated page.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 2

```
config.devServer = config.devServer || {};
```

Browser layout/build setting: config.devServer = config.devServer || {};; sizes the page or configures the browser development environment.

Line 3

```
config.devServer.headers = {
```

Browser layout/build setting: config.devServer.headers = {; sizes the page or configures the browser development environment.

Line 4

```
...(config.devServer.headers || {}),
```

Browser layout/build setting: ...(config.devServer.headers || {}),; sizes the page or configures the browser development environment.

Line 5

```
'Cross-Origin-Opener-Policy': 'same-origin',
```

Browser layout/build setting: 'Cross-Origin-Opener-Policy': 'same-origin',; sizes the page or configures the browser development environment.

Line 6

```
'Cross-Origin-Embedder-Policy': 'require-corp'
```

Browser layout/build setting: 'Cross-Origin-Embedder-Policy': 'require-corp'; sizes the page or configures the browser development environment.

Line 7

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.